



ESCUELA DE INGENIERÍA DE FUENLABRADA

INGENIERÍA TELEMÁTICA

TRABAJO FIN DE GRADO

TÍTULO DEL TRABAJO CON LETRAS MAYÚSCULAS
PARA SUSTANTIVOS Y ADJETIVOS

Autor : Alejandro Tejero de la Morena

Tutor : Dr. David Moreno Lumbreras

Curso académico 2025/2026



©2025 Alejandro Tejero de la Morena

Algunos derechos reservados

Este documento se distribuye bajo la licencia

“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,

disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

*Dedicado a
mi familia / mi abuelo / mi abuela*

Agradecimientos

Aquí vienen los agradecimientos... Aunque está bien acordarse de la pareja, no hay que olvidarse de dar las gracias a tu madre, que aunque a veces no lo parezca disfrutará tanto de tus logros como tú... Además, la pareja quizás no sea para siempre, pero tu madre sí.

Resumen

Aquí viene un resumen del proyecto. Ha de constar de tres o cuatro párrafos, donde se presente de manera clara y concisa de qué va el proyecto. Han de quedar respondidas las siguientes preguntas:

- ¿De qué va este proyecto? ¿Cuál es su objetivo principal?
- ¿Cómo se ha realizado? ¿Qué tecnologías están involucradas?
- ¿En qué contexto se ha realizado el proyecto? ¿Es un proyecto dentro de un marco general?

Lo mejor es escribir el resumen al final.

Summary

Here comes a translation of the “Resumen” into English. Please, double check it for correct grammar and spelling. As it is the translation of the “Resumen”, which is supposed to be written at the end, this as well should be filled out just before submitting.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Estructura de la memoria	3
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	6
2.3. Planificación temporal	6
3. Estado del arte	9
3.1. Backend	10
3.1.1. Python	10
3.1.2. Django	11
3.1.3. Base de datos: SQLite y PostgreSQL	12
3.2. Inteligencia artificial y modelos de lenguaje	12
3.2.1. Modelos de lenguaje y IA generativa	13
3.2.2. Ingeniería de prompts	14
3.2.3. Proveedores de modelos	15
3.3. Automatización e infraestructura	16
3.3.1. n8n	17
3.3.2. Docker	18
3.4. Frontend	19
3.4.1. HTML, CSS y JavaScript	19

3.4.2. Tailwind CSS	19
4. Diseño e implementación	21
4.1. Arquitectura general del proyecto	21
4.2. Sprint 1: Definición del proyecto	23
4.2.1. Tecnologías evaluadas	24
4.3. Sprint 2: Comienzo del desarrollo	25
4.3.1. Modelo de datos	25
4.3.2. Ingestión de datos	25
4.3.3. Detección de la colección principal	26
4.3.4. Sugerencia automática	27
4.3.5. Elección pagina	28
4.3.6. Calidad del mapping	28
4.3.7. Tests	28
4.4. Fase 3: Integración LLM y n8n	29
4.4.1. Conexión con LLM	30
4.4.2. Planner schema	31
4.4.3. Protección anti-alucinaciones	32
4.4.4. Actualización de modelos	34
4.4.5. Integración de n8n	35
4.5. Fase 4: Generador de proyectos Django	36
4.5.1. Arquitectura del generador	36
4.5.2. Fallbacks de generación	38
4.5.3. Evolución del modelo de datos	38
4.5.4. Dockerfile, endpoint y requirements	39
4.5.5. Workflow de despliegue n8n	40
4.6. Fase 5: Refactor del proyecto y consolidación del generador	42
4.6.1. Refactor del generador en módulos	43
4.6.2. Refactor de templates y ficheros de estilos	44
4.6.3. Enriquecimiento de prompts	47
4.6.4. Trazabilidad: GenerationLog	47

<i>ÍNDICE GENERAL</i>	<i>XI</i>
4.6.5. Verificación de consistencia	48
4.6.6. Panel de métricas	48
4.7. Fase 6: Funcionalidades avanzadas	49
4.7.1. Nuevos formatos de entrada: CSV y GeoJSON	49
4.7.2. Ejemplos few-shot por tipo de sitio	50
4.7.3. Versionado de proyectos generados	50
4.7.4. Usuarios del proyecto	51
4.7.5. LLM local y catálogo de modelos disponibles	51
4.8. Fase 7: Pulido y cierre	53
4.8.1. Sistema de presets visuales	53
4.8.2. Migración a PostgreSQL	54
4.8.3. Workflow generation-done	54
4.8.4. Workflow health-check	55
4.8.5. Workflow auto-shutdown	55
4.8.6. Mejora del workflow de despliegue	55
5. Experimentos y validación	57
5.0.1. Metodología de validación	57
5.0.2. Casos de uso	58
5.0.3. Análisis de errores	58
5.1. Métricas del sistema	60
5.1.1. Resumen general	61
5.1.2. Actividad y patrones de uso	61
5.1.3. Rendimiento de los modelos LLM	62
5.1.4. Alertas y estabilidad	63
5.1.5. Usuarios y proyectos generados	64
6. Resultados	67
7. Conclusiones	69
7.1. Consecución de objetivos	69
7.2. Aplicación de lo aprendido	71

7.3. Lecciones aprendidas	72
7.4. Trabajos futuros	73
A. Manual de usuario	75
Bibliografía	77

Índice de figuras

4.1. Arquitectura general del sistema WebBuilder	23
4.2. Sistema de scoring para la detección de la colección principal	27
4.3. Schema generado automáticamente por el LLM a partir de la API analizada . .	32
4.4. Flujo de ingestión y análisis: de URL a schema	34
4.5. Los 7 pasos del generador de proyectos Django	37
4.6. Workflow de despliegue n8n	42
4.7. Estructura de módulos del generador tras el refactor	43
5.1. Panel de Resumen general del sistema	61
5.2. Panel de actividad y patrones de uso	62
5.3. Panel de rendimiento de los modelos LLM	63
5.4. Panel de alertas del sistema	64
5.5. Panel de metricas sobre usuarios	64
5.6. Panel de historial de generaciones de la plataforma	65

Capítulo 1

Introducción

En estos últimos años, los modelos de lenguaje (LLMs) están cambiando nuestra relación con la tecnología. Herramientas como ChatGPT o Gemini que aparecieron hace un tiempo en la industria, o herramientas más actuales como Claude con gran potencial y variedad de uso han demostrado que en la mayoría de casos, hacer un buen uso de ellas puede sustituir tareas que tradicionalmente requerían conocimientos técnicos avanzados.

Este cambio ha creado la necesidad de actualizarse y aprender el manejo adecuado de estas herramientas, porque no todo es bueno como parece, pero con un buen uso y manejo de ellas se abren nuevas posibilidades en diferentes campos, como la redacción de textos, la traducción automática, la ayuda en generación de código, o como en este caso la generación automática de código para proyectos.

1.1. Contexto

En la actualidad, el desarrollo de webs es una tarea altamente demandada por todo tipo de sectores. Sin embargo, construir una aplicación desde cero requiere conocimientos técnicos sobre varios ámbitos: lenguajes de programación, bases de datos, frameworks, protocolos de comunicación, despliegue en producción, seguridad y diseño visual, entre otras. Esto implica que existen un gran número de usuarios que tienen ideas y datos interesantes pero no tienen los conocimientos necesarios para llevarlas a una aplicación funcional.

Además, hay gran cantidad de datos accesibles públicamente a través de APIs, ofreciendo datos sobre gran variedad de temas: catálogos de productos, datos meteorológicos, criptomo-

nedas, información geográfica, deportiva, etc. La mayoría de estos datos están perfectamente estructurados y disponibles de forma gratuita, pero requieren conocimientos técnicos para ser parseados, entendidos y presentados de forma útil al usuario final.

WebBuilder surge en la intersección de estas dos realidades. Por un lado, aprovecha los modelos de lenguaje para automatizar la generación del código que conecta los datos con la presentación visual. Por otro lado, ofrece una vía accesible para que cualquier usuario, independientemente de su nivel técnico, pueda transformar una fuente de datos pública en un sitio web funcional y desplegable.

1.2. Motivación

La motivación para realizar este proyecto surge de tres áreas diferentes pero capaces de juntarse en un único proyecto.

En primer lugar, la experiencia previa con Django, framework utilizado en la asignatura de Servicios Telemáticos y en el cual me sentí muy cómodo. La facilidad de comienzo de una nueva aplicación web más la cantidad de funcionalidades y personalización que ofrece, junto con la facilidad de organización de un proyecto grande hizo que fuese uno de los requisitos para empezar con esta base.

En segundo lugar, la inteligencia artificial. En este campo no me refiero a simplemente utilizarla como usuario final, sino a entender cómo funciona por dentro y aprender a controlar estas herramientas de forma adecuada. Un uso superficial de los modelos de lenguaje, sin comprender qué está ocurriendo realmente, genera una dependencia que puede volverse en contra del desarrollador en el momento menos esperado. Aprender prompt engineering correctamente, saber interpretar las respuestas del modelo y conocer sus limitaciones reales permite avanzar mucho más rápido y con criterio, siempre partiendo de una base técnica sólida sobre el problema que se quiere resolver.

En tercer lugar, otro área en la que estaba interesado es la automatización de procesos, queriendo aprender los conocimientos necesarios para poder crear flujos de trabajo automáticos. En este caso, crear procesos automáticos hace que todo sea más sencillo y rápido, pudiendo no solo lanzar proyectos con un simple click, sino también poder entender de forma actualizada como está funcionando tu proyecto por dentro con datos y métricas reales, teniendo de esta

forma control real sobre que sucede.

1.3. Estructura de la memoria

Esta memoria se organiza en siete capítulos, cada uno enfocado en un aspecto diferente del proyecto:

El **Capítulo 2** recoge los objetivos generales y específicos del proyecto, junto con la planificación temporal seguida durante el desarrollo y el resumen de las reuniones mantenidas con el tutor.

El **Capítulo 3** describe el estado del arte, revisando las tecnologías, herramientas y conceptos que han hecho posible la construcción de WebBuilder: el framework web, los modelos de lenguaje, las herramientas de automatización e infraestructura y las tecnologías de frontend.

El **Capítulo 4** aborda el diseño e implementación del sistema, describiendo el desarrollo por sprints desde la definición del proyecto hasta el cierre, con especial atención a las decisiones técnicas tomadas en cada sprint y los problemas encontrados, así como imágenes de como se encontraba el proyecto en cada sprint.

El **Capítulo 5** presenta los experimentos y la validación del sistema, incluyendo los casos de uso reales generados con la plataforma, el análisis de los errores detectados durante el desarrollo y las métricas del sistema obtenidas tras varias semanas de uso.

El **Capítulo 6** recoge los resultados obtenidos, evaluando en qué medida el sistema cumple con los requisitos planteados y mostrando los sitios generados como evidencia del funcionamiento real de la plataforma.

El **Capítulo 7** cierra la memoria con las conclusiones del proyecto, incluyendo la consecución de objetivos, la aplicación de los conocimientos del grado, las lecciones aprendidas durante el desarrollo y las posibles líneas de trabajo futuro.

Capítulo 2

Objetivos

2.1. Objetivo general

El objetivo principal del proyecto es el diseño e implementación de WebBuilder, una plataforma web capaz de generar automáticamente sitios web Django funcionales y desplegables a partir de los datos de cualquier API pública. El usuario introduce únicamente la URL de la API y una descripción en lenguaje natural de cómo quiere que sea el sitio, y el sistema se encarga del resto: analiza la estructura de los datos, propone un esquema de contenido, genera el código completo del proyecto y lo despliega en un contenedor Docker accesible mediante una URL de previsualización.

Para lograr este objetivo, WebBuilder integra tres tecnologías principales de forma coordinada. El backend Django actúa como núcleo del sistema, orquestando la ingestión de datos, la comunicación con los modelos de lenguaje y la generación del código. Los modelos de lenguaje de gran tamaño (LLM) son los responsables de analizar los datos de la API y producir el código Django adaptado a cada caso. Por último, n8n gestiona de forma asíncrona y las tareas de mayor duración, como el despliegue de los contenedores Docker o el envío de notificaciones al usuario.

El resultado es un sistema que permite a cualquier usuario, independientemente de sus conocimientos técnicos, obtener un sitio web personalizado, funcional y desplegable a partir de una fuente de datos pública en cuestión de minutos.

2.2. Objetivos específicos

El desarrollo de WebBuilder plantea una serie de objetivos específicos para las tareas principales en las que se ha dividido el trabajo. Estos objetivos recogen las funcionalidades concretas que el sistema debe cumplir y sirven como criterio de evaluación del grado de consecución del proyecto, retomados más adelante en el capítulo de conclusiones.

- **Desarrollar un módulo de ingestión de datos** capaz de validar, descargar y procesar el contenido de cualquier API pública en los formatos más habituales: JSON, XML, CSV y GeoJSON.
- **Integrar modelos de lenguaje** para el análisis automático de la estructura de los datos y la generación del código fuente de los proyectos Django, incluyendo modelos, vistas, plantillas e infraestructura de despliegue.
- **Implementar un generador de proyectos Django** que produzca, a partir del esquema validado por el usuario, un proyecto Django completo, funcional y desplegable en siete pasos secuenciales coordinados por el LLM.
- **Automatizar el despliegue de los proyectos generados** mediante contenedores Docker orquestados a través de flujos de trabajo en n8n, de forma que el usuario reciba una URL de previsualización del proyecto generado.
- **Diseñar un sistema de trazabilidad y métricas** que registre cada llamada al LLM durante la generación, detecte inconsistencias en el código producido y proporcione un panel de análisis para evaluar el rendimiento del sistema.
- **Ofrecer soporte multi-LLM y configuración personalizada**, permitiendo al usuario elegir entre un catálogo de modelos disponibles o conectar su propio proveedor mediante una clave de API cifrada.

2.3. Planificación temporal

El desarrollo de WebBuilder se ha llevado a cabo a lo largo de varios meses, compaginando siempre el proyecto con las clases presenciales de la universidad y trabajo durante algunos

meses. La dedicación ha sido progresiva y desigual a lo largo del tiempo. Los primeros meses, julio y agosto de 2025, se dedicaron principalmente a la investigación de tecnologías y la implementación de la base funcional de la aplicación.

Durante septiembre y octubre el avance fue más limitado, ya que la mayor parte del tiempo disponible se destinó a las últimas asignaturas del grado, aunque se aprovechó para mejorar la parte visual de la aplicación y consolidar la base existente. Entre diciembre y enero el proyecto estuvo prácticamente parado debido al periodo de exámenes, lo que supuso un parón necesario pero que retrasó el desarrollo. El trabajo realmente intensivo comenzó en febrero de 2026, una vez finalizados los exámenes. A partir de ese momento la dedicación fue diaria, entre siete y ocho horas, salvo un par de días a la semana reservados para las clases presenciales. Esta fase concentró la mayor parte del desarrollo técnico del proyecto y fue donde se tomaron las decisiones más importantes de arquitectura e implementación.

En cuanto a las reuniones con el tutor, se celebraron un total de xxx a lo largo del proyecto, manteniéndose un contacto periódico desde abril de 2025 hasta el cierre del desarrollo. A continuación se resume el contenido de cada reunión:

- **Reunión 1 (Abril 2025):** Primera toma de contacto con el tutor. Se presentaron dos ideas iniciales, ambas descritas mas adelante la sección 4.2
- **Reunión 2 (Julio 2025):** Tras varios meses de reflexión, se acordó la propuesta definitiva del proyecto: una aplicación Django capaz de procesar los datos de cualquier API pública y generar automáticamente un sitio web personalizado. El enfoque era más amplio y con mayor potencial que las ideas anteriores.
- **Reunión 3 (Julio 2025):** Se presentó la propuesta formal al tutor mediante un correo detallando las fases del sistema. Tras su aprobación se fueron implementando poco a poco.

El tutor propuso incorporar un módulo de inteligencia artificial que sugiriera y generara el código Django de forma automatizada, lo que supuso el punto de inflexión más importante del proyecto y definió su enfoque final hacia la generación automática asistida por LLM.

- **Reunión 4 (Octubre 2025):** Revisión de los avances realizados durante el verano y definición del rumbo del desarrollo. Se acordó una vez tenida la base elaborar un guion

detallado de los procesos que debería seguir la plataforma para garantizar un desarrollo ordenado y bien estructurado.

- **Reunión 5 (Octubre 2025):** Se presentó y revisó la lista de procesos definidos: validación de la URL introducida por el usuario, detección del formato de los datos devueltos por la API, análisis de la estructura del dataset y generación de un esquema preliminar. El tutor validó el enfoque y se continuó con el desarrollo.
- **Reunión 6 (Febrero 2026):** Tras el parón de exámenes, se retomó el contacto con el tutor para verificar el estado del proyecto y aclarar dudas sobre el proceso de documentación y la elección de base de datos antes de comenzar la fase más intensa. El tutor me sugirió integrar Inteligencia Artificial como motor principal del análisis en lugar del sistema de reglas heurístico, decisión que marcaría el rumbo del desarrollo a partir de ese momento.
- **Reunión 7 (Febrero 2026):** Se mostraron los primeros avances significativos del proyecto: el asistente de IA funcionando correctamente con el sistema de mapping, y n8n integrado con los primeros workflows de notificación por correo al registrarse e iniciar sesión. Se resolvieron dudas sobre la base de datos y se validó el camino seguido.
- **Reunión 8 (Marzo 2026):** Reunión previa a Semana Santa para mostrar los avances antes del descanso. En ese momento ya existían las primeras generaciones de páginas reales, aunque la mayoría presentaban errores frecuentes derivados de las alucinaciones del LLM y de la complejidad del generador en sus primeras versiones.
- **Reunión 9 (Abril 2026):** Última reunión del proyecto, centrada principalmente en la memoria y en los últimos flecos de implementación pendientes. Se definieron los puntos finales a pulir antes de la entrega y se estableció el calendario de cierre del proyecto.

Capítulo 3

Estado del arte

En este capítulo se describirán las tecnologías, herramientas y conceptos mas importantes que hicieron posible la creacion de WebBuilder, es decir, todo aquello que no ha sido creado en el marco de este trabajo pero que son fundamentales.

Cubriremos desde el framework web y la base de datos que sustentan la aplicacion, hasta los modelos de lenguaje y las herramientas de automatizacion e infraestructura que hacen posible el funcionamiento del proyecto.

El capítulo se organiza en cuatro bloques temáticos, recogidos en la Tabla 3.1.

Bloque	Tecnologías	Rol en el proyecto
Backend y base de datos	Python, Django, PostgreSQL	Núcleo de la aplicación: lógica de servidor, modelos y persistencia
IA y modelos de lenguaje	LLMs, Prompt Engineering, Groq, OpenRouter	Análisis de datos y generación automática de código
Automatización e infraestructura	n8n, Docker	Orquestación de flujos asíncronos y despliegue de proyectos
Frontend	HTML/CSS/JS, Tailwind CSS	Interfaz de la plataforma y presentación de los sitios generados

Cuadro 3.1: Bloques temáticos del estado del arte

3.1. Backend

El backend esta construido con diferentes tecnologias de desarrollo web profesional. En esta seccion se describen las herramientas que conforman la capa de servidor y persistencia de datos de la aplicacion.

3.1.1. Python

Python es un lenguaje de programación interpretado, de alto nivel y útil para diferentes tipos de tareas. Creado por Guido van Rossum a principios de los años 90 [13], Python ha evolucionado hasta convertirse en uno de los lenguajes más utilizados del mundo, con un 57,9 % de uso entre los desarrolladores en 2025 [16], especialmente en el ámbito del desarrollo web, el análisis de datos y la inteligencia artificial. Siendo uno de los lenguajes de programacion mas versatiles

La sintaxis clara permite escribir programas concisos para combinar logica del servidor, procesamiento de APIs y generación dinámica de código. Python cuenta con un ecosistema de bibliotecas muy completo, distribuido a través de PyPI (*Python Package Index*), del que WebBuilder hace uso directo: `requests` para la descarga de datos desde APIs externas, `xmltodict` y `defusedxml` para el parseo seguro de XML y `zipfile` para el empaquetado de los proyectos generados.

Durante todo el proyecto, mas alla del resto de tecnologias mencionadas a continuación, Python actua como base durante toda la logica del servidor, durante gran parte de procesos:

- **Ingestión de datos:** descarga y procesamiento del contenido devuelto por APIs externas en los formatos soportados por la plataforma (JSON, XML, CSV y GeoJSON).
- **Validación de datos:** comprobación del formato y la accesibilidad de la URL introducida por el usuario, así como la verificación de que los datos devueltos son parseables y contienen información útil.
- **Comunicación con modelos de lenguaje:** construcción y envío de los prompts al LLM, gestión de la respuesta y extracción del contenido estructurado necesario para cada fase del asistente.

- **Generación dinámica de código:** producción programática de todos los ficheros que componen el proyecto Django generado, incluyendo modelos, vistas, plantillas, configuración de despliegue y scripts de carga de datos.

La versión utilizada es Python 3, aprovechando características modernas del lenguaje como las *f-strings* (cadenas de texto con expresiones Python embebidas, usadas en la construcción de prompts para el LLM).

3.1.2. Django

Django es un framework de desarrollo web de código abierto para Python, que promueve el desarrollo rápido y escalable, así como un diseño limpio [1]. Fue publicado en 2005 y sigue el principio “*batteries included*”: incluye de serie todo lo necesario para construir aplicaciones web completas, desde el sistema de enrutado de URLs hasta el ORM (*Object-Relational Mapper*), el motor de plantillas, el sistema de autenticación y el panel de administración.

Permite configurar todo lo necesario, Django incluye un sistema de gestión de base de datos **SQLite** integrado que permite definir las tablas y sus campos directamente en Python, sin necesidad de escribir SQL. Cada tabla de la base de datos se representa como una clase, y Django se encarga de traducir automáticamente las operaciones sobre esos objetos (consultas, inserciones, actualizaciones) a las instrucciones SQL correspondientes. Esto facilita el desarrollo inicial, y también la evolución del modelo a través de migraciones automáticas, que registran cada cambio de forma controlada, siendo capaz de saber que se cambió o modificó en cada momento, teniendo más control sobre el desarrollo.

La lógica de vistas y de URLs permite organizar la lógica de forma clara y separada, con funciones clave que responden peticiones HTTP, separando el procesamiento de datos de la presentación. El motor de plantillas permite generar HTML dinámico con una sintaxis sencilla, con soporte para herencia de plantillas, lo que favorece la reutilización de componentes visuales.

En WebBuilder, Django gestiona el ciclo de vida completo de cada análisis:

- **Recepción de la URL:** recibe y valida la URL de la API introducida por el usuario a través del asistente.
- **Ingestión y parseo:** coordina la descarga del contenido de la API y su procesamiento en el formato correspondiente (JSON, XML, CSV o GeoJSON).

- **Invocación del módulo LLM:** envía los datos procesados al cliente de lenguaje y gestiona la respuesta para cada fase del asistente.
- **Persistencia:** almacena en base de datos el estado del análisis, los resultados obtenidos y el historial de generaciones por usuario.
- **Interfaz de usuario:** sirve las vistas del asistente paso a paso, el historial, las métricas y las páginas de administración.

La versión empleada es Django 5.1, que introduce mejoras en el ORM (capa de software que permite trabajar con la base de datos usando objetos Python en lugar de escribir SQL) y en el sistema de migraciones respecto a versiones anteriores.

3.1.3. Base de datos: SQLite y PostgreSQL

SQLite es un sistema de base de datos ligero que almacena toda la información en un único fichero local, sin necesidad de instalar ni configurar un servidor externo [15]. Esta simplicidad lo convierte en la opción por defecto de Django para entornos de desarrollo, aunque presenta limitaciones en escrituras concurrentes al utilizar bloqueos sobre el fichero de datos.

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto, robusto y ampliamente utilizado en entornos de producción [18]. Ofrece soporte completo para transacciones ACID, concurrencia real mediante MVCC (*Multi-Version Concurrency Control*) y un rico conjunto de tipos de datos, lo que lo convierte en una opción adecuada cuando se requiere mayor robustez y escalabilidad.

En WebBuilder se utilizaron ambas tecnologías de forma progresiva: SQLite durante las fases iniciales de desarrollo por su sencillez, y PostgreSQL en las fases finales para soportar un entorno de producción más exigente. El detalle de esta migración se describe en el capítulo 4. El detalle de la migración de SQLite a PostgreSQL y los motivos técnicos que la justificaron se describen en la Sección 4.8.2.

3.2. Inteligencia artificial y modelos de lenguaje

La inteligencia artificial generativa constituye el núcleo funcional del proyecto. En esta sección se introducen los conceptos fundamentales sobre los modelos de lenguaje, el prompting

engineering y los proveedores de API utilizados para acceder a los diferentes modelos.

3.2.1. Modelos de lenguaje y IA generativa

Los modelos de lenguaje de gran tamaño (LLM, *Large Language Models*) son sistemas de inteligencia artificial entrenados sobre grandes volúmenes de datos. Su arquitectura se basa en redes neuronales de tipo *Transformer* [19], introducidas en 2017, que sustituyeron el procesamiento secuencial del texto por un mecanismo de atención capaz de analizar todas las palabras simultáneamente y capturar relaciones de largo alcance entre ellas.

A diferencia de los sistemas tradicionales de procesamiento de lenguaje natural, los LLM no se programan con reglas explícitas, sino que aprenden a predecir la siguiente unidad de texto, denominada token, a partir de las anteriores. Un token no tiene porque ser necesariamente una palabra completa, dependiendo del modelo y del idioma puede representar una palabra entera o un fragmento de ella. Así es como se da lugar a modelos capaces de mantener conversaciones, redactar textos, traducir idiomas o como en este caso, generar código funcional a partir de instrucciones en lenguaje natural. Cada modelo tiene una ventana de contexto llamada **context window**, que limita el número máximo de tokens que puede procesar simultáneamente, incluyendo tanto el prompt de entrada como la respuesta generada. Superar este límite implica que el modelo pierde información de los mensajes más antiguos. Este límite es muy importante, ya que los prompts de generación incluyen el esquema de datos, ejemplos del dataset e instrucciones de formato, lo que puede consumir varios miles de tokens por llamada, como se detalla en el capítulo 4.

Un fenómeno frecuente en los LLM son las alucinaciones, estas ocurren cuando el modelo genera contenido aparentemente válido pero en realidad es incorrecto o directamente inexistente [21]. A diferencia de un error de compilación, una alucinación no produce un fallo sino una salida que parece válida pero que contiene datos inventados, nombres de campos que no existen en el dataset o código erróneo. En WebBuilder las alucinaciones son el principal riesgo del sistema, ya que un campo inventado en el modelo Django generado provoca errores silenciosos en tiempo de ejecución. El diseño de los mecanismos de protección descritos en la Sección 4.4.3 responden directamente a este problema.

La irrupción mediática de modelos como ChatGPT en 2022 [9] marcó el inicio de una etapa de adopción masiva de la IA generativa. Desde entonces han aparecido numerosos modelos,

tanto comerciales (GPT-4, Claude, Gemini) como de pesos abiertos (LLaMA, Mistral, DeepSeek), accesibles a través de APIs públicas o ejecutables localmente. Esta diversidad permite a WebBuilder elegir el modelo más adecuado para cada tarea en función de criterios como coste, latencia o calidad de respuesta.

En este trabajo, los LLM no se utilizan como simples chatbots, sino como componentes programables dentro de un flujo automatizado: el modelo recibe datos estructurados, devuelve análisis razonados y produce código fuente que se integra directamente en el proyecto generado.

3.2.2. Ingeniería de prompts

La ingeniería de prompts (*prompt engineering*) es la disciplina que estudia cómo formular las instrucciones enviadas a un modelo de lenguaje para obtener respuestas precisas y acorde a una necesidad, intentando a su vez reducir el número de alucinaciones. A diferencia de la programación tradicional, donde el comportamiento del sistema se especifica mediante código, en los LLM el comportamiento se controla a través del lenguaje natural del propio prompt.

Para poder diseñar un buen prompt debemos seguir y combinar varios elementos:

- **Rol del modelo:** una descripción clara de la identidad o función que debe asumir el modelo durante la tarea.
- **Instrucciones detalladas:** una especificación precisa de lo que se espera que el modelo haga y cómo debe hacerlo.
- **Formato de respuesta:** indicaciones explícitas sobre la estructura que debe tener la salida, como JSON, código Python o texto plano.
- **Restricciones:** limitaciones explícitas que acotan el comportamiento del modelo y reducen la probabilidad de respuestas incorrectas o fuera de contexto.
- **Ejemplos de referencia:** muestras del estilo de salida esperado que guían al modelo hacia el resultado deseado. Esta técnica, conocida como *few-shot prompting*, ha demostrado mejorar notablemente la calidad de las respuestas en tareas complejas o con formatos estrictos [11].

La calidad del resultado generado por un LLM depende, en **gran medida**, de la precisión del prompt. Un prompt ambiguo puede provocar respuestas inconsistentes, alucinaciones o salidas que no respetan el formato esperado. Por ello, el diseño, la iteración y el refinamiento de los prompts constituye una parte central del trabajo de desarrollo en sistemas basados en LLM, comparable en importancia a la propia escritura del código que los invoca.

La calidad del resultado no depende únicamente de la precisión del prompt, sino también de la capacidad del modelo utilizado. Un modelo con poco entrenamiento o potencia insuficiente producirá respuestas pobres independientemente de la calidad del prompt. Ambos factores, diseño del prompt y elección del modelo, son por tanto determinantes en el resultado final.

En el contexto de WebBuilder, la ingeniería de prompts juega un papel crítico en dos fases: el análisis de los datos de la API y la generación del código Django del proyecto. Cada uno de estos prompts ha sido diseñado tras innumerables pruebas para producir resultados coherentes y de calidad profesional, como se detallará en el capítulo de diseño e implementación. Además, durante el proyecto se fueron guardando los datos de cada generación para realizar un estudio exhaustivo de que funciona mejor para cada caso.

3.2.3. Proveedores de modelos

Para acceder a un LLM y poder usarlo, en la mayoría de casos es necesario recurrir a un proveedor de API que disponga del modelo y ofrezca su uso. Cada vez existen más proveedores en el mercado, cada uno con su propio catálogo. La gran mayoría de estos proveedores siguen el estándar de OpenAI descrito a continuación, lo que permite conectar de forma sencilla con WebBuilder. Cabe aclarar que los modelos gratuitos de los proveedores mencionados a continuación realizan bien su trabajo, pero nunca de forma perfecta. Las alucinaciones a día de hoy siguen existiendo sobre todo en los modelos gratuitos.

Groq [3] es un proveedor especializado en inferencia de baja latencia. Utiliza hardware propio (LPU, *Language Processing Units*) optimizado, lo que le permite ofrecer velocidades de generación muy superiores a las de proveedores tradicionales basados en GPU. Groq aloja modelos de pesos abiertos como LLaMA, y dispone de un nivel de uso gratuito con límites de peticiones por minuto y por día. Su baja latencia hace que sea la opción preferente, donde la generación de templates es rapidísima.

OpenRouter [12] es un proveedor que da acceso a decenas de modelos de distintos provee-

dores a través de una única API. Lo mejor de este proveedor es la sencillez para la conexión: una sola clave de API permite consultar modelos de OpenAI, Anthropic, Google, Meta o cualquier otro proveedor integrado en la plataforma. Esto facilita las pruebas comparativas entre modelos y reduce el coste de cambiar de proveedor en caso necesario.

NVIDIA NIM [8] (*NVIDIA Inference Microservices*) es el servicio de ejecución de modelos en la nube de NVIDIA. Ofrece acceso a más de cien modelos de distintos fabricantes, entre ellos Meta, Mistral, DeepSeek y los propios modelos Nemotron de NVIDIA, ejecutados sobre infraestructura de GPUs de alto rendimiento. Al igual que los proveedores anteriores, expone sus modelos a través de una API compatible con el formato OpenAI, lo que permite integrarlo en WebBuilder sin ninguna modificación en el cliente LLM. Su inclusión en el catálogo de proveedores de la plataforma amplía las opciones disponibles para el usuario, que puede elegir entre modelos optimizados para generación de código, razonamiento o tareas de propósito general.

Los tres proveedores descritos comparten el denominado formato *OpenAI-compatible*, un estándar surgido a partir de la API pública de OpenAI [10]. Este estándar define un único endpoint `/chat/completions` que acepta una lista de mensajes con roles diferenciados (`system`, `user` y `assistant`) y devuelve la respuesta del modelo en un formato JSON uniforme. Su adopción masiva por parte de los principales proveedores del mercado lo ha convertido en el protocolo común de la industria de modelos de lenguaje.

Esto es algo muy importante para el proyecto, al construir el cliente LLM sobre este protocolo común, el sistema puede cambiar de proveedor o de modelo modificando únicamente tres parámetros de configuración (la URL base, la clave de API y el identificador del modelo), sin necesidad de modificar ninguna línea de la lógica interna. Esta decisión de diseño, detallada en la Sección 4.4.1, es la que hace posible que WebBuilder soporte múltiples proveedores de forma nativa y que cualquier usuario pueda conectar su propio modelo personalizado.

3.3. Automatización e infraestructura

WebBuilder no solo se apoya en el backend de Django y Python, también se usan herramientas externas para la automatización de tareas y el despliegue de los proyectos. En esta sección se describen las herramientas de n8n, encargado de orquestar los flujos asíncronos de la

plataforma, y de Docker, que proporciona el entorno aislado en el que se ejecutan los proyectos generados.

3.3.1. n8n

n8n [7] es una plataforma de automatización de flujos de trabajo de código abierto que permite conectar aplicaciones, APIs y servicios mediante una interfaz visual basada en nodos. Cada flujo de trabajo (*workflow*) está compuesto por una secuencia de nodos que se ejecutan en cadena, donde cada nodo representa una acción, explicando su función dentro del contenido del mismo nodo, hay varias funciones:

- **Disparadores:** eventos que inician el flujo, como una petición HTTP entrante (webhook) o un temporizador programado (cron).
- **Peticiones HTTP:** llamadas a APIs externas o internas para intercambiar datos entre servicios.
- **Transformación de datos:** procesamiento y manipulación de la información entre nodos mediante lógica personalizada en JavaScript.
- **Envío de correos:** notificaciones automáticas por email a usuarios o administradores ante determinados eventos del sistema.
- **Ejecución de scripts:** bloques de código personalizado que permiten implementar lógica compleja directamente dentro del flujo.
- **Consultas a base de datos:** lectura o escritura de datos en sistemas de persistencia externos integrados en el workflow.

Varios de los nodos implementados en este proyecto ejecutan código JavaScript personalizado, lo que permite implementar lógicas complejas directamente dentro del flujo, como la gestión y despliegue de contenedores Docker.

En WebBuilder, n8n actúa como una capa desacoplada que gestiona todo lo que no forma parte del ciclo síncrono de petición-respuesta de Django: el envío de correos automáticos al registrarse o iniciar sesión, la notificación al finalizar una generación, la monitorización diaria

del estado del sistema, el apagado automático de contenedores inactivos y, sobre todo, el despliegue de los proyectos generados en contenedores Docker. Esta arquitectura permite añadir nuevas automatizaciones sin modificar el código del backend principal.

A diferencia de plataformas similares, n8n puede autoalojarse, lo que permite ejecutarlo en infraestructura propia y mantener el control completo sobre los datos y las credenciales, como ocurre en este proyecto donde se ejecuta en un contenedor Docker local.

La comunicación entre WebBuilder y n8n se basa en el patrón *webhook*: una llamada HTTP que un sistema realiza hacia otro para notificar que ha ocurrido un evento, sin esperar una respuesta inmediata. A diferencia de una llamada síncrona, donde el proceso queda bloqueado hasta recibir el resultado, un webhook permite que el sistema emisor continúe su ejecución mientras el receptor procesa la tarea de forma independiente [14].

3.3.2. Docker

Docker [2] es una herramienta que permite empaquetar aplicaciones junto con sus dependencias en unidades aisladas llamadas contenedores. A diferencia de la virtualización tradicional, los contenedores comparten el núcleo del sistema operativo anfitrión, lo que los hace mucho más ligeros y rápidos de arrancar que una máquina virtual convencional.

Cada contenedor se construye a partir de una imagen, un artefacto inmutable que contiene el sistema de ficheros, el software instalado y la configuración necesaria para ejecutar la aplicación. Las imágenes se definen mediante un fichero de texto llamado *Dockerfile*, que describe paso a paso cómo construir el entorno: la imagen base, los paquetes a instalar, el código a copiar y el comando que se ejecuta al arrancar el contenedor.

Esta capacidad de empaquetado garantiza que la aplicación se ejecute de forma idéntica en cualquier entorno: el equipo de desarrollo, un servidor de pruebas o un entorno de producción. Eliminando los problemas derivados de las diferencias entre los entornos de desarrollo y producción.

En WebBuilder, Docker cumple dos funciones diferenciadas. Por un lado, aloja la propia herramienta de n8n utilizada por la plataforma de forma local. Por otro, y más importante, cada proyecto Django generado por WebBuilder incluye automáticamente su propio *Dockerfile*, lo que permite desplegarlo en un contenedor aislado sin necesidad de instalar dependencias en el sistema anfitrión. Un flujo específico de n8n (*webbuilder-deploy*), construye la imagen, lanza

el contenedor y le asigna un puerto disponible de forma totalmente automática.

3.4. Frontend

WebBuilder tiene dos capas de frontend diferenciadas: la interfaz de la propia plataforma, construida con tecnologías web estándar, y los proyectos que genera.

En este primer caso, la interfaz de WebBuilder está construida con HTML, CSS y JavaScript, las tres tecnologías fundamentales de la web, estandarizadas por el W3C (*World Wide Web Consortium*) [20]. HTML define la estructura de cada página, CSS controla su presentación visual y JavaScript gestiona la interactividad en el lado del cliente.

3.4.1. HTML, CSS y JavaScript

HTML [5] (*HyperText Markup Language*) es el lenguaje de marcado que define la estructura y el contenido de las páginas web. En WebBuilder, todas las plantillas Django están escritas en HTML, organizando los elementos visuales del asistente, los formularios, las tablas de historial y los paneles de información.

CSS [4] (*Cascading Style Sheets*) es el lenguaje de estilos que controla la presentación visual del HTML: colores, tipografías, espaciados, disposición de los elementos y diseño responsive. La interfaz de WebBuilder utiliza CSS para definir tanto el tema visual general de la plataforma como los modos claro y oscuro implementados durante el desarrollo.

JavaScript [6] es el lenguaje de programación que añade interactividad en el lado del cliente, permitiendo modificar el contenido de la página sin necesidad de recargarla. En WebBuilder se emplea en varias vistas clave: la previsualización dinámica del plan propuesto por el LLM, el editor de código integrado para inspeccionar los ficheros generados y la actualización del estado del asistente en cada paso del flujo.

3.4.2. Tailwind CSS

Tailwind CSS [17] es un framework de hojas de estilo basado en el enfoque *utility-first*. A diferencia de frameworks tradicionales como Bootstrap, que ofrecen componentes predefinidos (botones, tarjetas, formularios) listos para usar, Tailwind proporciona un conjunto extenso de

clases que se combinan directamente en el HTML para construir cualquier diseño desde cero.

Este enfoque ofrece varias ventajas prácticas. En primer lugar, evita el problema de los estilos huérfanos y la acumulación de CSS no utilizado, ya que cada clase aplicada está explícitamente presente en el marcado. En segundo lugar, permite un control fino sobre el diseño sin necesidad de sobrescribir estilos predefinidos, lo que reduce la complejidad del CSS final. Por último, su sistema de diseño coherente con escalas predefinidas para espaciados, colores, tipografías y puntos de ruptura responsivos favorece la consistencia visual entre componentes.

	Tailwind CSS	Bootstrap	CSS puro
Forma de trabajar	Se aplican clases pequeñas directamente en el HTML	Se usan componentes visuales ya contruidos	Se escriben estilos manualmente en ficheros .css
Personalización	Total, sin restricciones	Limitada por el diseño de cada componente	Total, sin restricciones
Curva de aprendizaje	Media	Baja	Alta
Estilos sin usar	Mínimos, solo se carga lo que se usa	Elevados, se carga el framework completo	Depende del desarrollador
Generación por LLM	Natural y consistente, ampliamente representado en los datos de entrenamiento	Inconsistente, el modelo tiende a mezclar versiones	Impredecible, el modelo inventa nombres de clase sin referencia

Cuadro 3.2: Comparativa entre los principales enfoques de estilos CSS

Tailwind se ha convertido en una de las opciones más populares para proyectos web modernos [16], y su naturaleza declarativa encaja especialmente bien con la generación automática de interfaces por parte de modelos de lenguaje, ya que los LLM tienden a producir HTML con clases Tailwind de forma natural gracias a la abundancia de ejemplos presentes en sus datos de entrenamiento.

En WebBuilder, todos los proyectos Django generados incluyen Tailwind CSS como sistema de estilos por defecto, garantizando una apariencia visual coherente y profesional sin necesidad de mantener hojas de estilo personalizadas para cada proyecto generado.

Capítulo 4

Diseño e implementación

4.1. Arquitectura general del proyecto

Antes de comenzar a detallar los diferentes sprints donde se explica el desarrollo de cada avance, se presenta la arquitectura final del sistema, para que el lector pueda entender el flujo completo y como fue desarrollandose.

Como se muestra en la Figura 4.1, el sistema se divide en tres bloques principales, descritos a continuación.

El primero es **Django**, que actúa como núcleo de la aplicación. Dentro de este bloque, el módulo de ingestión se encarga de validar la URL introducida por el usuario, descargar el contenido de la API de forma segura y detectar automáticamente el formato de los datos, ya sea JSON, XML, CSV o GeoJSON. A continuación, el módulo de análisis LLM procesa la estructura del dataset y propone un schema con los campos más relevantes, que el usuario puede revisar y modificar antes de continuar. Una vez aceptado el plan, el generador produce un proyecto Django completo en siete pasos secuenciales, generando desde los modelos y vistas hasta los templates HTML y los ficheros de infraestructura necesarios para el despliegue. Todo el proceso queda registrado en la base de datos PostgreSQL, que almacena tanto los modelos de la aplicación como los logs de cada llamada al LLM para facilitar la depuración. Finalmente, el módulo de notificaciones informa al usuario en los momentos clave del flujo: al registrarse, al iniciar sesión y cuando su sitio ha sido generado correctamente.

El segundo bloque es **n8n**, que gestiona las automatizaciones más costosas en tiempo y que no pueden ejecutarse de forma síncrona desde Django. En total se desarrollaron seis workflows,

agrupados en dos categorías. Por un lado, los workflows de **autenticación y notificación**: *Register* envía un correo de bienvenida al usuario al registrarse; *Login* notifica al usuario cada vez que inicia sesión; y *generation-done* le informa cuando su sitio ha sido generado correctamente. Por otro lado, los workflows de **infraestructura**: *deploy* recibe la señal de Django mediante un webhook, construye la imagen Docker del proyecto generado, arranca el contenedor en un puerto libre y verifica que el sitio ha quedado operativo antes de devolver la URL de preview; *health-check* comprueba diariamente que el servidor sigue activo y alerta al administrador si detecta algún problema; y *auto-shutdown* apaga automáticamente los contenedores que llevan inactivos más de un tiempo configurable, liberando recursos del servidor sin intervención manual.

El tercer bloque son los **servicios externos**, que el sistema consume pero no controla. El proveedor de LLM, ya sea Groq, OpenRouter, NVIDIA o un modelo configurado por el propio usuario, recibe los prompts del módulo de análisis y del generador y devuelve las respuestas en texto plano. Las APIs públicas son la fuente de datos que el usuario introduce en el sistema y de las que Django descarga el contenido a procesar. El servidor de correo recibe las notificaciones de n8n y las entrega al usuario final. Por último, el host Docker es el entorno donde se ejecutan los contenedores de los sitios generados, accesibles mediante una URL de preview.

La comunicación entre Django y n8n se realiza exclusivamente mediante webhooks, de forma que ambos sistemas están desacoplados y un fallo en las automatizaciones nunca bloquea el funcionamiento principal de la aplicación.

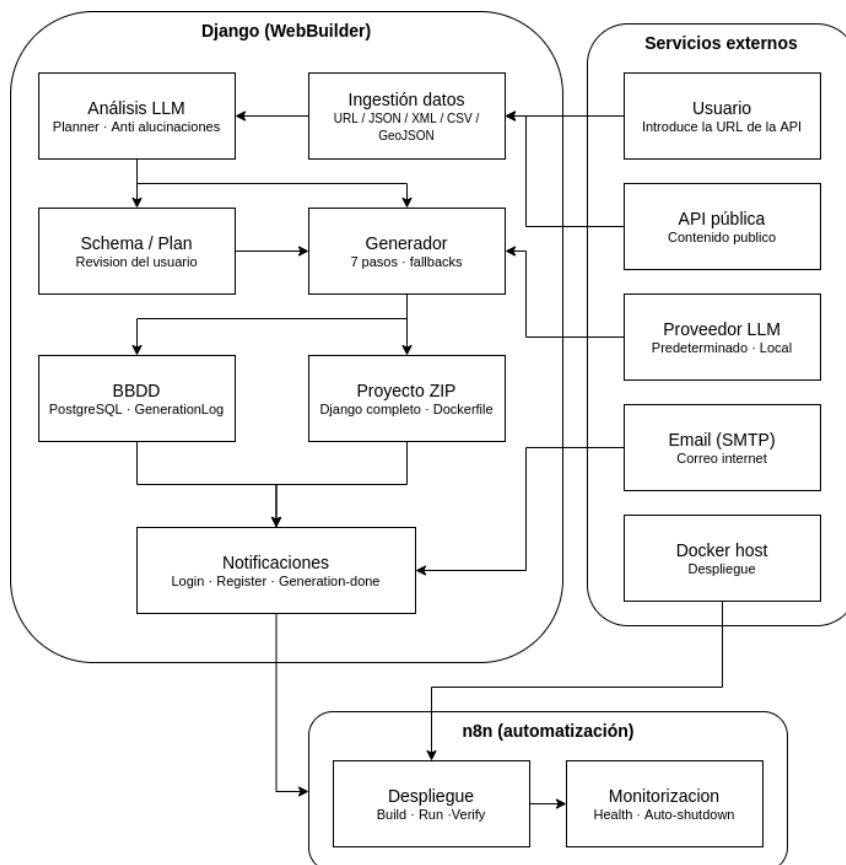


Figura 4.1: Arquitectura general del sistema WebBuilder

4.2. Sprint 1: Definición del proyecto

El punto de partida del proyecto fue una reunión con el tutor en abril de 2025, en la que se presentaron posibles ideas al tutor. En esa primera toma de contacto surgieron un par de ideas más, las cuales finalmente se acabaron descartando en nuestra segunda reunión, ya que decidimos adaptarnos un poco más a la situación actual de las páginas webs y automatizaciones.

Reunión	Fecha	Resultado
1ª	Abril 2025	Presentación de ideas iniciales y definición del enfoque
2ª	Julio 2025	Selección del proyecto final: WebBuilder

Cuadro 4.1: Reuniones de definición del proyecto

La primera idea consistía en una aplicación web que, a partir de datos de APIs hospitalarias,

ayudara a los pacientes a encontrar el centro médico más adaptado a sus necesidades según una serie de parámetros. La segunda idea ampliaba esa primera hacia un asistente de mudanza. Esta segunda idea planteaba un escenario sobre un usuario que deseaba instalarse en una ciudad, la aplicación recogería todos los datos de las APIs públicas y un parseo de los datos proporcionados por agencias y otras empresas, de tal forma que introduciendo el usuario sus requisitos, se le organizaría un posible plan de vida en la zona. Ambas ideas tenían en común la lectura y procesamiento de APIs públicas, pero el alcance era demasiado limitado, lo que limitaba el valor del proyecto.

En la segunda reunión, realizada en Julio de 2025, llegamos a una idea más acotada, una aplicación web Django encargada de procesar los datos de cualquier API pública y generar automáticamente un sitio web personalizado. Más tarde pensé en posibles nombres para el proyecto, hasta que se acordó el nombre de WebBuilder.

Durante este período fui recogiendo información y creando las posibles fases del sistema: validación de la URL, procesamiento de los datos recogidos, preferencias sobre el diseño, etc. El tutor valoró positivamente estos pasos, lo que me impulsó para seguir adelante con el proyecto.

Durante los meses de verano el proyecto comenzó poco a poco creando las partes más sencillas para ir dándole forma: la página de inicio, el sistema de login y registro de usuarios. Aunque en el futuro la parte estética cambiaría, la base más o menos se mantendría.

Al principio no hubo un gran desarrollo de código, sino que se dedicó este período a definir con más detalle el flujo de la aplicación, se investigaron tecnologías y se diseñaron procesos que debería seguir el sistema. Este trabajo previo fue muy importante para el correcto desarrollo guiado de la aplicación. Estas tecnologías se explican más en detalle en la siguiente subsección.

4.2.1. Tecnologías evaluadas

Se eligió **Django** como framework backend por su ORM integrado y su compatibilidad con las bibliotecas de procesamiento de datos necesarias, así como la fácil adaptación y escalabilidad de Python.

Para la automatización de flujos asíncronos se consideró la implementación **n8n** por su capacidad de autoalojamiento y su interfaz visual, aunque su incorporación al proyecto se realizaría más tarde.

Sobre la integración de LLMs, no se planteó desde el inicio como una posibilidad, aunque

más adelante como se describe en la sección 4.4 se realizaría su implementación.

4.3. Sprint 2: Comienzo del desarrollo

Con la propuesta del proyecto ya definida, el desarrollo comenzó con fuerza en febrero de 2026. Poco a poco se fue creando la estructura base de la aplicación: modelos, sistema de ingestión de datos, lógica de análisis, y una interfaz mínima para poder hacer pruebas del flujo completo. La arquitectura hasta este punto era bastante sencilla: una aplicación Django que recibía una URL, descargaba los datos de la API, los analizaba y devolvía una propuesta de estructura para el futuro sitio web.

4.3.1. Modelo de datos

El único modelo de la base de datos en este sprint era `APIRequest`, que recogía los datos de cada petición URL realizada. Sus campos principales se muestran en la Tabla 4.2. Este modelo fue el eje central durante este sprint y será la base del análisis durante todo el proyecto.

Campo	Tipo	Descripción
url	CharField	URL de la API introducida por el usuario
raw_data	TextField	Datos en crudo descargados de la API
parsed_data	JSONField	Estructura de datos ya parseada
status	CharField	Estado del análisis: pending, processed o error
field_mapping	JSONField	Mapping resultante mostrado al usuario
created_at	DateTimeField	Fecha y hora de creación de la petición
user	ForeignKey	Usuario propietario del análisis

Cuadro 4.2: Campos del modelo `APIRequest` en la Sprint 2

4.3.2. Ingestión de datos

El modulo de ingestión de datos, ubicado en `utils/ingest/` era y será el encargado de validar la URL introducida por el usuario, realizar la descarga del contenido de forma segura y detectar automáticamente el formato de los datos. La validación comprobaba que la URL

procedería de `http` o `https`, que fuese procedente de un dominio válido y que su longitud no superase los 2048 caracteres, límite establecido para evitar ataques mediante URLs maliciosas por su longitud. La descarga se realizaba en modo streaming, es decir, en vez de esperar que el contenido del archivo se descargue entero, se va leyendo el contenido en fragmentos, cada uno con un límite de 1MB, y un timeout de 8 segundos. Conseguimos de esta forma evitar que una API con respuestas excesivamente grande nos tumbe o bloquee el sistema.

La detección de formato se realiza inspeccionando los primeros caracteres del contenido de la API, de esta forma se conseguía distinguir entre JSON y XML, que son los contenidos manejados hasta este momento, mas adelante en el capítulo 4.7.1 se aumenta el tipo de datos a procesar. En cualquier caso siempre se devuelve una estructura Python homogénea, que se utiliza en adelante para el sistema, de forma que no es necesario seguir trabajando con el formato original.

4.3.3. Detección de la colección principal

La parte mas compleja de este sprint fue entender la estructura interna de los campos de las distintas APIs. Los campos de las APIs no suelen ser una lista plana de items en la raiz, es decir, la colección principal no esta expuesta, sino que esta envuelta bajo claves de metadatos como `results`, `data` o `items`, o incluso con varios niveles de profundidad.

Para resolver esto se implementó un nuevo modulo: `utils/analysis`, donde se crearon ficheros con funciones importantes como `find_main_items`, la cual recorría recursivamente la estructura de la API, y se evaluaba con un sistema de puntuación propio para sacar la estructura principal.

Cada lista candidata se puntuaba según cinco factores:

- **Densidad de diccionarios:** porcentaje de elementos de la lista que son diccionarios. Las listas con menos de un 55 % de dicts recibían una penalización proporcional, ya que era poco probable que representaran una colección de entidades reales.
- **Tamaño:** listas con más items recibían mayor puntuación, aunque con un límite para evitar que una lista enorme pero trivial se impusiera sobre una colección más pequeña pero más relevante.

- **Consistencia de claves:** si todos los diccionarios de la lista compartían aproximadamente las mismas claves, era muy probable que representaran entidades del mismo tipo. Se calculaba como el promedio de claves propias de cada ítem sobre la unión de todas las claves observadas.
- **Bonus por claves semánticas:** si los campos de la colección incluían nombres típicos de contenido como `title`, `name`, `id`, `description` o `image`, se añadía una bonificación proporcional al número de coincidencias.
- **Penalización por profundidad:** los paths muy anidados o que pasaban por índices numéricos se penalizaban para favorecer colecciones cercanas a la raíz, que solían ser las más relevantes.

Al finalizar este scoring, se seleccionaba la lista con mayor puntuación como colección principal, junto con la ruta dentro de la estructura original. En la Figura 4.2 se muestra cómo los cinco factores se combinan en la función de scoring para obtener la colección principal.

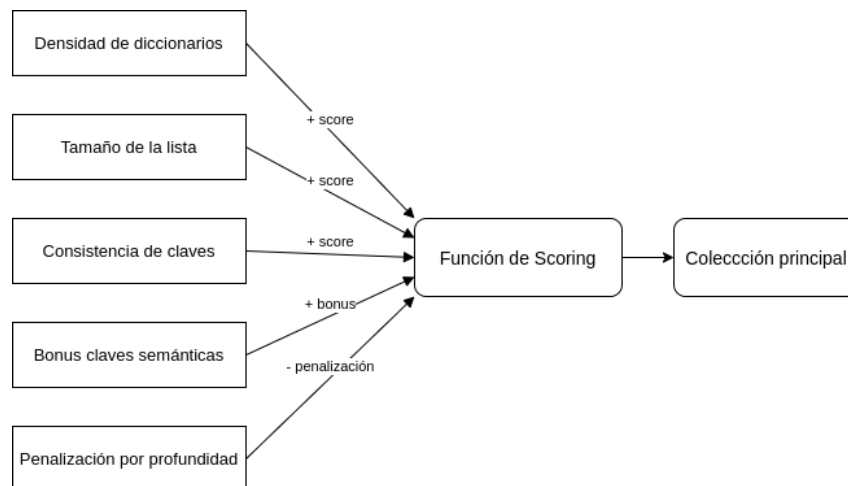


Figura 4.2: Sistema de scoring para la detección de la colección principal

4.3.4. Sugerencia automática

Una vez localizada la colección principal, se realizó la implementación del nuevo fichero `utils/mapping.py`, encargado de sugerir automáticamente qué campos del dataset debían ocupar cada rol: fecha, id, descripción, etc. Para ello se utilizaba un catálogo predefinido que asociaba cada rol con sus nombres más habituales. Por ejemplo un campo llamado `published_at`,

obtenía un alto score para el rol fecha ya que contenía la subcadena `published`. Además de la comparación por nombre, el sistema añadía score extra si el tipo del campo observado coincidía con el tipo esperado en el rol.

Para evitar que la misma clave del dataset se asignara como primera sugerencia de varios roles a la vez, se implementó una función llamada `suggest_mapping`, que aplicaba una asignación por orden de prioridad: primero se reservaba la mejor clave para el rol `id`, luego para `title`, luego para `description`, y así sucesivamente. Esto reducía la repetición de sugerencias y hacía el mapping resultante más útil para el usuario, sin embargo, ocurrían errores como que algunos campos se quedasen vacíos o que al asignarse por orden de prioridad, algunos campos sueltos se asignasen a roles que no tenían relacion.

4.3.5. Elección pagina

Para guiar mejor al usuario durante el proceso de mapping, se añadió un sistema de perfiles de intención en `intents.py`. El usuario podía indicar si quería construir un blog, un portfolio, un catálogo o un directorio, y el sistema reordenaba los roles y ajustaba cuáles eran obligatorios, recomendados u opcionales según ese tipo de sitio.

4.3.6. Calidad del mapping

Además, se implementó un sistema de puntuación de calidad del mapping en `mapping.py`, que evaluaba de 0 a 100 el mapping introducido por el usuario teniendo en cuenta si había asignado los roles más importantes, si existían duplicados entre roles críticos y qué campos quedaban vacíos. El resultado se mostraba al usuario con una etiqueta cualitativa: *Excelente*, *Bueno*, *Aceptable* o *Mejorable*.

4.3.7. Tests

Desde el primer sprint se escribieron tests para los cuatro módulos principales del sistema heurístico, organizados en ficheros independientes: `test_analysis`, `test_mapping`, `test_normalizer` y `test_parsers`. Estos test cubrían casos como la detección de colecciones anidadas, la asignación de roles por prioridad, la normalización de valores y el parseo

seguro de XML. Esto permitió verificar el comportamiento del sistema de scoring con datasets variados, detectando casos en los que las heurísticas fallaban antes de que llegaran al usuario.

Limitaciones del enfoque heurístico

Conforme se probaba el sistema con datasets reales muy distintos entre sí, los límites del enfoque heurístico se fueron haciendo evidentes. Una API de plantas medicinales, una de personajes de ficción y una de productos de e-commerce tienen nomenclaturas de campos completamente diferentes, y un sistema de reglas fijas no podía cubrir todos los casos con la misma fiabilidad. Cuando los nombres de los campos no coincidían con los campos de la API, el mapping era genérico y malo para el usuario. El sistema funcionaba razonablemente bien con APIs convencionales y con campos estándar, pero fallaba ante cualquier dataset con nombres o estructura poco habituales. Esta limitación fue la que motivó el cambio de enfoque descrito en el siguiente sprint.

4.4. Fase 3: Integracion LLM y n8n

Con la fase 2 terminada y viendo que las limitaciones eran evidentes, se tomó la decisión de realizar un cambio fundamental para el futuro del proyecto. Se abandonó la idea del sistema de reglas, aunque no en la totalidad, ya que algunas partes nos seguirían siendo útiles. Se decidió integrar un modelo de lenguaje (LLM) como núcleo o motor del análisis y generación de las páginas. Hasta este momento el sistema intentaba entender los datos mediante puntuaciones y comparaciones. A partir de este punto, se decidió que fuese la Inteligencia Artificial la encargada de escoger y proponer la estructura del sitio y decidiera que campos mostrar. Como se comentó anteriormente, no todo el flujo de análisis e ingesta se cambió, se eliminaron las partes de score y propuesta de dataset, pero el parseo y análisis de los campos se mantuvo y modificó levemente, se introdujo todo el dataset en una estructura, que sería la que se pasaría al LLM, de forma que evitaríamos alucinaciones o errores del LLM en la lectura de la API, obteniendo de esta forma un schema más consistente. Este proceso se describe con más detalle en la subsección 4.4.3

4.4.1. Conexión con LLM

Para poder realizar la conexión con el LLM, el primer paso fue crear el módulo `utils/llm`, y dentro se creó el archivo `client.py`, el cual implementa la función `chat_completions`. Esta función se encarga de hacer una única cosa: enviar un mensaje al LLM y devolver la respuesta en texto plano.

Una decisión importante fue utilizar el estándar de la mayoría de proveedores para realizar la conexión, utilizando de esta forma el endpoint `/chat/completions`. En este caso creado para poder realizar pruebas con varios LLM de diferentes proveedores (OpenRouter, Groq, NVIDIA) y cualquier agente local. Otra decisión fue crear `.env`, de forma que la URL base, la API key y el modelo se leen de variables de entorno definidas en dicho fichero, lo que permite cambiar de proveedor sin tocar una sola línea de código.

La función mencionada anteriormente acepta tres parámetros principales:

- **Mensaje de usuario:** la tarea concreta que se le envía al modelo en cada llamada, en este caso el prompt con los datos del dataset, las claves disponibles y los ejemplos de items reales.
- **Mensaje de sistema:** instrucciones previas que definen cómo debe comportarse el modelo antes de procesar el mensaje del usuario. Se utiliza para indicarle el rol que debe adoptar, el formato de respuesta esperado y lo que está prohibido hacer. Es opcional, ya que técnicamente todo puede ir en el mensaje de usuario, pero separarlo es una buena práctica que facilita reutilizar el mismo comportamiento base en distintas llamadas. Por ejemplo, el mensaje de sistema del planner indica al modelo que es un experto en diseño de sitios web, que debe devolver únicamente un objeto JSON válido y que está prohibido inventar campos que no existan en el dataset proporcionado.
- **Temperatura:** parámetro que controla el grado de aleatoriedad en las respuestas del modelo. Un valor cercano a 0 produce respuestas más deterministas y predecibles, mientras que valores más altos generan respuestas más creativas pero menos consistentes. En este caso se fija por defecto en `0.2`, favoreciendo la estabilidad especialmente cuando se espera JSON estructurado como salida.

El primer proveedor utilizado fue OpenRouter, con el modelo de meta-llama:

`llama-3.3-70b-instruct:free`, de acceso gratuito y potencia más que suficiente para las primeras pruebas.

4.4.2. Planner schema

Otro fichero clave fue `utils/llm/planner.py` y su función `generate_site_plan`. La misión era transformar el resultado del análisis en un propuesta con campos concretos de schema que el usuario pudiese aceptar y modificar a su gusto antes de continuar.

El schema tiene una forma fija que el resto del sistema espera:

- **site.type**: tipo de sitio recomendado, entre `blog`, `portfolio`, `catalog`, `dashboard` u `other`.
- **site.title**: nombre descriptivo del sitio propuesto por el modelo.
- **fields**: lista ordenada de campos del dataset que el sitio debe mostrar, cada uno con su clave original y una etiqueta legible para el usuario.

Para obtener este esquema, se construye un prompt estructurado que se envía al LLM. Este prompt es el resultado de una serie de pruebas con distintas formulaciones, siguiendo técnicas de *prompt engineering* [11] para conseguir respuestas consistentes y parseables. La información contenida en este prompt es la siguiente:

- **Objetivo del usuario**: texto en lenguaje natural introducido por el usuario describiendo qué tipo de sitio quiere construir.
- **Claves disponibles**: lista de campos reales extraídos del dataset durante la fase de análisis. El modelo solo puede proponer campos que estén en esta lista.
- **Ejemplos del dataset**: hasta tres items reales del dataset, para que el modelo pueda entender qué tipo de valores contiene cada campo y proponer etiquetas adecuadas.
- **Contrato JSON**: estructura exacta que debe devolver el modelo, con los campos obligatorios y su formato esperado.
- **Reglas de formato**: conjunto de instrucciones explícitas que indican al modelo que debe devolver únicamente un objeto JSON válido, sin bloques de código Markdown, sin texto introductorio y sin ningún contenido fuera del JSON.

Esta rigidez sobre el formato de salida es intencionada, ya que tras la realización de varias pruebas, los fallos generados por los diferentes LLM siempre eran parecidos. Como esa respuesta se usa directamente en el sistema, cualquier texto adicional rompe el proceso de parseo.

De todas formas, debido a las alucinaciones de algunos modelos y a que en ocasiones ignoraban las restricciones de formato, se aplicaba una limpieza previa sobre la respuesta antes de parsearla. Esta limpieza eliminaba bloques de código Markdown como ````json` o `````, que algunos modelos añaden automáticamente alrededor del JSON aunque se les indique explícitamente que no lo hagan, así como espacios y caracteres sobrantes al inicio y al final de la respuesta, los cuales rompían el flujo de ejecución del proyecto.

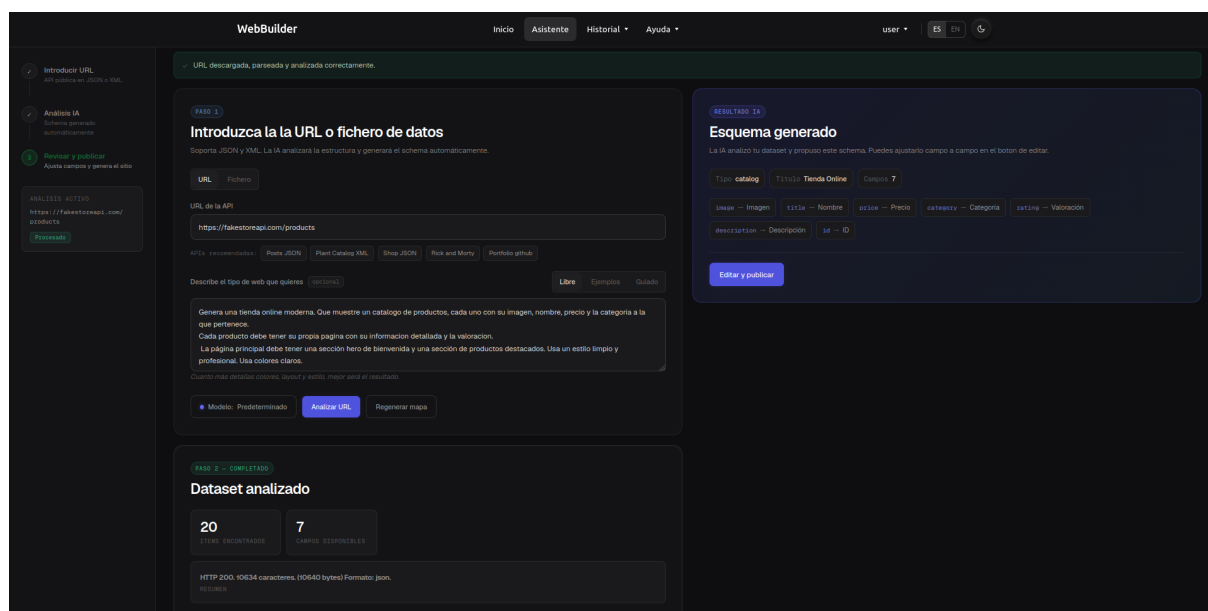


Figura 4.3: Schema generado automáticamente por el LLM a partir de la API analizada

4.4.3. Protección anti-alucinaciones

Los errores más comunes siempre sucedían por la misma causa, las alucinaciones del LLM. El error más común era inventar información que no existía en la API, y que nunca se parseo, por lo que no existe en el dataset construido. Para ello hubo que aplicar varias capas de defensa contra alucinaciones para proteger los schemas.

- **Validación contra claves reales:** cada clave propuesta por el LLM se comprueba contra la lista de claves reales extraída del dataset. Si la clave no existe, se intenta un match sin

distinguir mayúsculas y minúsculas. Si tampoco hay coincidencia, la clave se descarta silenciosamente.

- **Filtro de tokens prohibidos:** existe una lista de palabras que el modelo confunde con claves del dataset porque forman parte del propio prompt del usuario. Al estar palabras como `available_keys`, `fields` o `examples` presentes en el texto del prompt, algunos modelos las interpretan erróneamente como campos reales del dataset y las incluyen en su respuesta. Por ejemplo, si el prompt contiene la línea `^available_keys": ["name", "date", "url"]`, el modelo puede devolver `available_keys` como uno de los campos seleccionados, cuando en realidad es una etiqueta del propio prompt. Cualquier clave que coincida con esta lista se descarta automáticamente.
- **Detección de valores disfrazados de claves:** se detectan los casos en los que el modelo introduce un valor concreto del dataset donde debería haber puesto el nombre de un campo. Por ejemplo, si el dataset tiene un campo de fecha, el modelo puede devolver `2024-01-15` como clave en lugar de `published_at`. Para evitarlo, se descartan automáticamente cadenas que sean fechas en formato ISO, URLs completas, extensiones de imagen, valores nulos o texto excesivamente largo.
- **Fallback:** si tras aplicar todas las validaciones no queda ninguna clave válida, el sistema construye automáticamente un schema de emergencia con las primeras claves del dataset, garantizando que el flujo nunca se bloquea por un fallo del modelo.

Si el parseo falla, el sistema reintenta automáticamente una vez con una pista sobre el error cometido. Si el segundo intento también falla, se activa el fallback. En la Figura 4.4 se muestra el flujo completo desde que el usuario introduce la URL hasta obtener el schema validado.

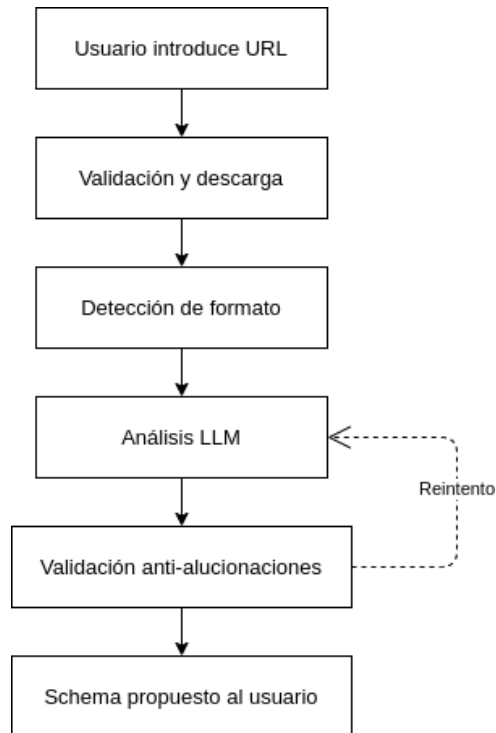


Figura 4.4: Flujo de ingestión y análisis: de URL a schema

4.4.4. Actualización de modelos

La integración del LLM trajo consigo dos cambios importantes.

El primero de ellos fue una modificación en el modelo existente **APIRequest**, añadiéndole el campo `plan_accepted`. Siendo este un booleano encargado de guardar si el usuario ha aceptado el schema propuesto antes de continuar.

El segundo fue la creación de un nuevo modelo **GeneratedSite**, vinculado con el modelo **APIRequest** mediante una relación `OneToOne`. Sus campos principales se muestran en la Tabla 4.3.

Campo	Tipo	Descripción
api_request	OneToOneField	Referencia al análisis que originó el sitio
public_id	UUIDField	Identificador único público del sitio generado
plan	JSONField	Schema aceptado por el usuario con los campos seleccionados
project_files	JSONField	Ficheros del proyecto generado indexados por ruta

Cuadro 4.3: Campos principales del modelo GeneratedSite en la Fase 3

4.4.5. Integración de n8n

En paralelo a la integración del LLM, durante esta fase se incorporó n8n al sistema por primera vez. Es una herramienta de automatización de flujos que permite conectar servicios mediante webhooks y nodos, y que más adelante sería una pieza fundamental de la arquitectura del proyecto. En este momento su uso fue todavía inicial: instalar el entorno y crear el primer flujo de pruebas.

n8n se instaló en un contenedor Docker siguiendo la imagen oficial, quedando disponible en `localhost:5678`. Una vez levantado el entorno, se creó el primer flujo en n8n y se configuró el primer webhook, apuntando a la ruta `/webhook-test/WebBuilder-Register`. El prefijo `webhook-test` indica que el webhook estaba en modo de prueba, el modo que n8n ofrece para verificar que el flujo funciona correctamente antes de activarlo en producción.

La integración en Django se realizó en `views/auth.py`, donde se añadió la función `llamar_webhook`. El nodo se encargaría de notificar a los usuarios recién registrados: Django realizaba una llamada POST a ese webhook con el nombre de usuario y el email del recién registrado, y n8n enviaba automáticamente un correo de bienvenida personalizado a esa dirección. Más adelante se completaría el flujo con el webhook de login.

La función estaba diseñada para ser completamente transparente al flujo principal: si n8n no estaba disponible o la llamada fallaba, la excepción se capturaba silenciosamente y el registro continuaba con normalidad. Esto garantizaba que un fallo en el sistema de notificaciones nunca pudiera bloquear el acceso a la aplicación.

4.5. Fase 4: Generador de proyectos Django

Tras la implementación del LLM y el sistema de schema funcionando correctamente, el siguiente paso era el más complejo del proyecto, ya que requería coordinar múltiples llamadas al LLM de forma secuencial garantizando que cada fichero generado fuese compatible con los anteriores. Dicho paso era convertir el schema aprobado por el usuario en un proyecto Django completo, funcional y desplegable. Durante esta fase del proyecto surgió el fichero más importante hasta la fecha: **utils/generator/project_generator**, el núcleo de WebBuilder.

4.5.1. Arquitectura del generador

El generador se organiza como un orquestador de siete pasos secuenciales, cada uno con su responsabilidad concreta y encargado de producir uno o varios ficheros del proyecto final. El LLM interviene en los primeros seis pasos, que se encargan de la creación de ficheros, pero en el séptimo paso ya no interviene, y se concluye la ejecución con el ensamblaje de todos los ficheros generados anteriormente.

- **Paso 1 — Estructura de páginas:** el LLM decide qué páginas tendrá el sitio, sus URLs, sus nombres de vista y si cada página es un listado, una página de detalle o una portada. El resultado es una lista de páginas que guiará todos los pasos siguientes.
- **Paso 2 — `models.py`:** el LLM genera el modelo de datos Django adaptado a los campos del dataset.
- **Paso 3 — `views.py` y `urls.py`:** el LLM genera las vistas necesarias para cada página decidida en el paso 1. El fichero `urls.py` se genera en Python directamente a partir de la lista de páginas, sin intervención del modelo.
- **Paso 4 — `base.html`:** el LLM genera la plantilla base del sitio, con la estructura HTML común a todas las páginas: cabecera, navegación y bloque de contenido.
- **Paso 5 — Templates por página:** el LLM genera un template HTML independiente para cada página, adaptado a su tipo y a los campos del dataset.
- **Paso 6 — `load_data.py`:** el LLM genera un comando de gestión de Django que, al ejecutarse, descarga los datos de la API original y los inserta en la base de datos del sitio

generado. Este fichero es fundamental para que el sitio tenga contenido real desde el primer arranque.

- **Paso 7 — Archivos fijos:** se ensamblan todos los ficheros de infraestructura que no dependen del dataset ni del LLM: `settings.py`, `manage.py`, `wsgi.py`, `requirements.txt`, `Dockerfile` y `entrypoint.sh`.

El resultado final es un diccionario Python donde cada clave es una ruta relativa de fichero y cada valor es su contenido. Este diccionario se guarda en el campo `project_files` del modelo `GeneratedSite`, comentado más adelante en la subsección 4.5.3

En la Figura 4.5 se muestra la secuencia completa de los siete pasos del generador.

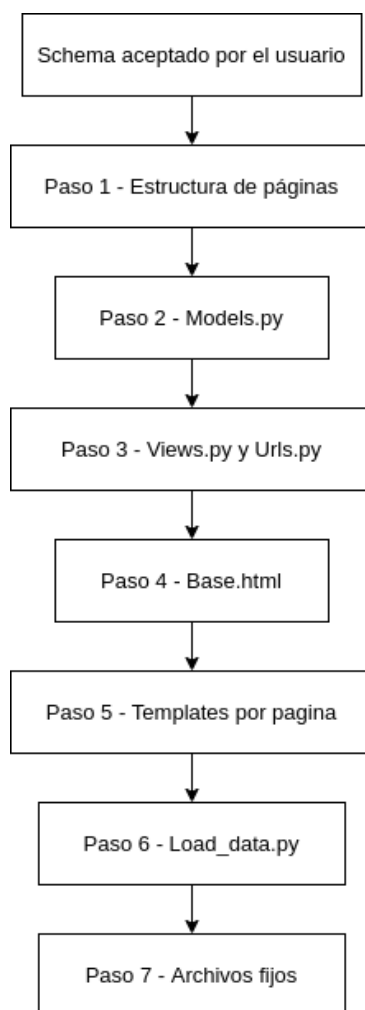


Figura 4.5: Los 7 pasos del generador de proyectos Django

4.5.2. Fallbacks de generación

Durante las pruebas de generación se comprobó que las alucinaciones del LLM eran constantes y que pulir cada prompt requeriría un trabajo considerable. El LLM cometía fallos en cualquiera de los pasos, devolviendo código vacío, con errores de sintaxis o código no adaptado al paso correspondiente. Para garantizar que el proyecto resultante fuese siempre desplegable, se añadió un fallback por cada paso del generador.

El funcionamiento era el siguiente: si la respuesta del LLM era vacía o inválida, el sistema activaba automáticamente una versión mínima pero funcional del fichero en cuestión. Por ejemplo, el fallback de **models.py** generaba un modelo genérico llamado `Item` con un campo `CharField` por cada campo del dataset. El fallback de **views.py** generaba vistas básicas de listado y detalle sobre ese mismo modelo. El fallback de **pages** definía tres páginas fijas: portada, catálogo y detalle. Y el fallback de **load_data.py** generaba un comando mínimo de carga de datos a partir de la URL original de la API.

De esta forma, aunque el LLM produjese un resultado de baja calidad en algún paso, el proyecto generado siempre sería desplegable, aunque con una interfaz más genérica.

4.5.3. Evolución del modelo de datos

El modelo `GeneratedSite` fue variando durante esta fase para reflejar los nuevos estados del sistema. Se añadieron los campos `project_files` para almacenar todos los ficheros generados, `project_name` con el nombre del proyecto y `preview_url` para guardar la URL del sitio desplegado (explicado en la subsección 4.5.5). Además se incorporaron dos nuevos campos de estado cuya evolución se muestra en la Tabla 4.4.

Campo	Valores	Significado
generation_status	pending	El sitio está en espera de ser generado
	generating	El generador está produciendo los ficheros
	ready	Los ficheros han sido generados correctamente
	error	La generación ha fallado en algún paso
deploy_status	idle	El contenedor no ha sido desplegado aún
	deploying	El workflow de n8n está construyendo la imagen
	done	El contenedor está en funcionamiento
	error	El despliegue ha fallado en algún paso

Cuadro 4.4: Estados del modelo GeneratedSite en la Fase 4

Esta separación entre el estado de generación y el estado de despliegue permitía saber con precisión en qué punto del proceso se encontraba cada sitio.

4.5.4. Dockerfile, entryptpoint y requirements

Cada proyecto generado incluía un `Dockerfile`, un script `entrypoint.sh` y un fichero `requirements.txt` que en conjunto automatizaban completamente el arranque del sitio. El objetivo era que cualquier proyecto generado por WebBuilder se desplegara mediante Docker pulsando un botón, sin configuración manual por parte del usuario.

El `Dockerfile` partía de la imagen oficial `python:3.12-slim`. Copiaba el `requirements.txt`, instalaba las dependencias con `pip` y copiaba el resto del proyecto. Hasta entonces el puerto expuesto era el 8000, sobre el que se serviría la aplicación, esto variaría más adelante con la mejora del flujo de despliegue de n8n.

El fichero `requirements.txt` incluido en cada proyecto generado contenía únicamente las dependencias mínimas para el funcionamiento del sitio: `django>=4.2, <6.0` como framework web, `requests>=2.28` para que el comando `load_data` pudiera descargar los datos de la API original, y `gunicorn>=21.0` como servidor de producción. La decisión de mantener las dependencias al mínimo fue intencionada: cuantas menos dependencias tenga el

proyecto generado, menos probabilidades hay de que el `docker build` falle por incompatibilidades o paquetes no disponibles.

El script `entrypoint.sh` era el responsable del arranque completo del sitio, los pasos que ejecutaba eran los siguientes:

1. **Generación de migraciones:** ejecutaba `python manage.py makemigrations siteapp` para crear los ficheros de migración a partir del `models.py` generado por el LLM, siendo `siteapp` el nombre de la aplicación interna del proyecto generado.
2. **Aplicación de migraciones:** ejecutaba `python manage.py migrate` para crear las tablas en la base de datos SQLite del contenedor.
3. **Carga de datos:** ejecutaba `python manage.py load_data`, el comando generado por el LLM en el paso 6 del generador, que descargaba los datos de la API original e insertaba los registros en la base de datos.
4. **Arranque del servidor:** lanzaba Gunicorn con dos workers y un timeout de 120 segundos para servir la aplicación Django en el puerto 8000.

Esta secuencia garantizaba que al acceder al sitio por primera vez ya estuviera completamente funcional: base de datos creada, datos cargados y servidor en marcha. El uso de Gunicorn en lugar del servidor de desarrollo de Django era una decisión deliberada para que los sitios generados tuvieran un nivel mínimo de robustez en producción.

4.5.5. Workflow de despliegue n8n

En paralelo al desarrollo del generador de archivos, se fue construyendo el workflow más importante del proyecto, encargado del despliegue de las páginas.

La decisión de delegar el despliegue en n8n y no en Django respondía a un problema técnico concreto: el proceso de build de Docker puede tardar varios minutos, y ejecutarlo de forma síncrona desde Django bloquearía el servidor durante ese tiempo. Al separarlo y crearlo con n8n, Django lo único que tiene que hacer es el enviar la señal para que comience la ejecución del workflow.

Esta señal se envía desde la ruta `/webhook/webbuilder-deploy`. Django realiza una llamada POST con el nombre del proyecto como parámetro y a partir de aquí el workflow se ejecuta siguiendo los siguientes nodos sucesivamente:

1. **Descomprimir ZIP:** el primer nodo crea el directorio de despliegue en `/files/deloys/<project_name>` y descomprime el ZIP del proyecto generado en un subdirectorio `src/`.
2. **Restablecer contenedor:** antes de construir la nueva imagen, el workflow intenta detener y eliminar cualquier contenedor anterior con el mismo nombre (`wb-<project_name>`). Si el contenedor no existe, el error se ignora y el flujo continúa sin interrupciones.
3. **Elegir puerto:** el nodo escanea los puertos actualmente en uso por Docker en el rango 8100–8999 y selecciona el primer puerto libre disponible. Si no hay ningún puerto libre en ese rango, el flujo lanza un error controlado.
4. **Docker Build:** ejecuta `docker build` sobre el directorio del proyecto con un timeout de 5 minutos. Si el build falla, captura los últimos 500 caracteres de los logs de error para facilitar el diagnóstico.
5. **Docker Run:** arranca el contenedor en modo *detached* con el nombre y el puerto asignados, mapeando el puerto elegido al puerto interno 8000 del contenedor.
6. **Verificar Docker:** espera 20 segundos para dar tiempo al `entrypoint.sh` a ejecutar las migraciones y cargar los datos, y luego comprueba que el contenedor sigue en funcionamiento. Si el contenedor ha caído, recupera las últimas 40 líneas de sus logs y lanza un error con esa información para facilitar el diagnóstico.
7. **Responder al webhook:** si todo ha ido bien, devuelve a Django un JSON con la URL del preview, el nombre del contenedor y el puerto asignado, que WebBuilder almacena en el campo `preview_url` del modelo `GeneratedSite`.

Este diseño garantiza que el usuario siempre recibe una respuesta clara: si el despliegue tiene éxito obtiene la URL del sitio generado, y si falla en cualquier paso Django es notificado con información suficiente para actualizar el estado del sitio a `error` y mostrar un mensaje al usuario explicando lo ocurrido.

En la Figura 4.6 se muestra el flujo completo del workflow de despliegue.

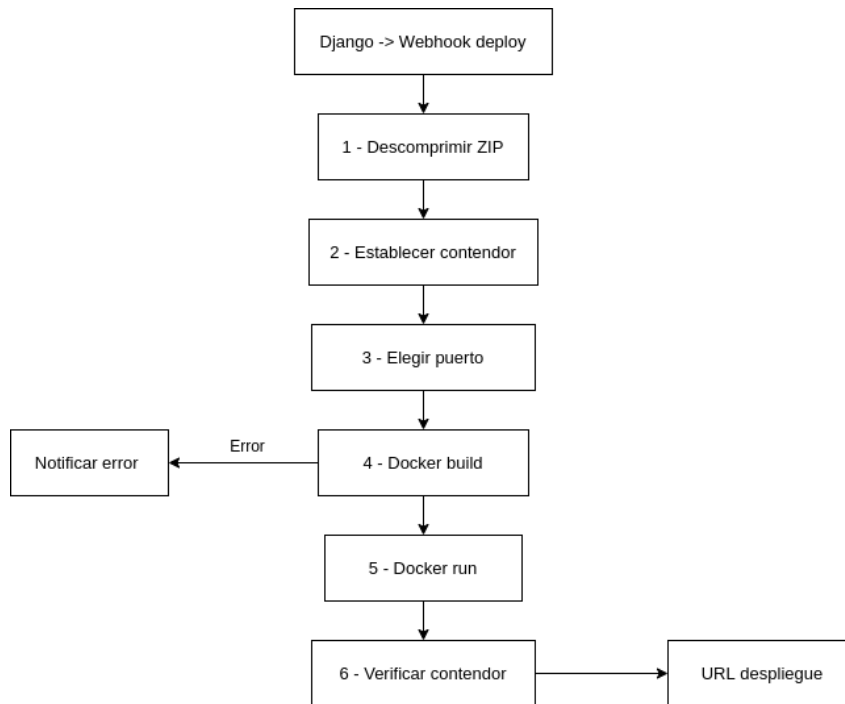


Figura 4.6: Workflow de despliegue n8n

4.6. Fase 5: Refactor del proyecto y consolidación del generador

Una vez realizadas las primeras pruebas del generador, el código del fichero **project_generator.py** había crecido considerablemente hasta convertirse en un módulo difícil de mantener y depurar. Esta situación causó un refactor profundo del proyecto, que no se limitó al generador sino que abarcó también los ficheros de presentación, el sistema de prompts y la trazabilidad de las llamadas al LLM.

Esta fase se dividió en dos bloques principales. En primer lugar, la refactorización del generador y de los ficheros de presentación, dividiendo el código en módulos más pequeños y mantenibles. En segundo lugar, la incorporación de nuevas funcionalidades de trazabilidad y análisis, como el modelo **GenerationLog**, el verificador de consistencia y el panel de métricas.

4.6.1. Refactor del generador en módulos

Estos cambios fueron muy importantes, los pasos del generador se realizaban con todo el contenido dentro del mismo fichero, algo que no era de fácil escalabilidad a futuro, por lo que el primer cambio fue dividir el **project_generator.py** en cuatro ficheros encargados cada uno de una labor específica dentro de **utils/generator**, como se muestra en la Figura 4.7.

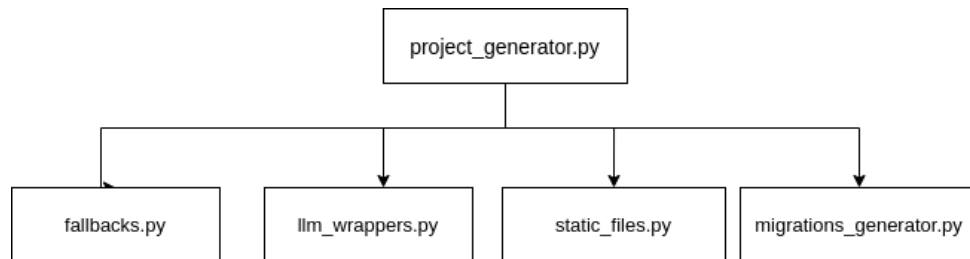


Figura 4.7: Estructura de módulos del generador tras el refactor

- **llm_wrappers.py**: Centralizó todas las llamadas al LLM en tres funciones reutilizables. `llm_call` para llamadas básicas que devuelven texto, `llm_json_call` para llamadas que esperan JSON como respuesta, y `llm_call_logged` para llamadas que además guardan un registro automático en base de datos con el prompt enviado, la respuesta recibida y los posibles errores, a través del modelo **GenerationLog** descrito en la subsección 4.6.4.
- **fallbacks.py**: Agrupó todos los fallbacks de emergencia en un único fichero. Cada función de este módulo generaba una versión mínima pero funcional de un artefacto concreto del proyecto: páginas, modelos, vistas o templates. Al tenerlos centralizados resultaba mucho más sencillo mejorarlos o ajustarlos sin tocar el orquestador principal.
- **static_files.py**: Extrajo toda la generación de ficheros de infraestructura que no dependen del LLM: `manage.py`, `settings.py`, `wsgi.py`, `urls.py`, `Dockerfile`, `entrypoint.sh` y `requirements.txt`. El orquestador principal quedó así mucho más limpio, delegando en este módulo la construcción del scaffolding fijo.
- **migrations_generator.py**: Aisló la lógica de generación automática de la migración inicial, que parseaba el `models.py` producido por el LLM y construía el fichero `0001_initial.py` correspondiente.

4.6.2. Refactor de templates y ficheros de estilos

La misma causa que creó la decisión del refactor del generador ocurrió con los ficheros de presentación del proyecto (HTML, CSS y JavaScript). Hasta el momento los templates HTML eran ficheros que incluían todas las secciones en un único bloque, al igual que ocurría con los ficheros CSS y JavaScript. Como se mencionó anteriormente, esta fase introdujo una reorganización completa de la interfaz con la separación de responsabilidades.

En cuanto a los templates se creó un nuevo módulo **partials**, encargado de contener los fragmentos que representan una sección concreta de cada página. Cada fichero principal pasó a contener sus partials, incluyéndose mediante la etiqueta **include** de Django.

La home del proyecto, que hasta entonces era el fichero más complejo, se dividió en seis partials independientes. En la Tabla 4.5 se muestra la estructura completa de partials tras el refactor:

Partial	Página	Responsabilidad
hero.html	home	Sección principal de bienvenida
examples.html	home	Ejemplos de sitios generados
how.html	home	Explicación del funcionamiento
llms.html	home	Modelos de lenguaje compatibles
metrics.html	home	Métricas públicas del sistema
cta.html	home	Botón de acceso a la aplicación
input.html	assistant	Formulario de introducción de URL
analysis.html	assistant	Resultado del análisis del dataset
preview.html	assistant	Schema propuesto por la IA
published_sites.html	history	Sitios generados y desplegados
recent_analysis.html	history	Análisis recientes del usuario

Cuadro 4.5: Partials HTML tras el refactor de templates

El asistente se dividió en tres partials dentro de `templates/WebBuilder/partials/assistant/`

- `input.html` — formulario del paso 1, donde el usuario introduce la URL de la API o sube un fichero local para su análisis
- `analysis.html` — resultado del paso 2, que muestra las estadísticas del dataset analizado: número de items encontrados, campos disponibles y ruta de la colección principal
- `preview.html` — resultado del paso 3, donde se muestra el schema propuesto por la IA y el usuario puede revisarlo y aprobarlo antes de continuar

El historial se dividió en dos partials dentro de `templates/WebBuilder/partials/history/`:

- `published_sites.html` — listado de sitios ya generados y desplegados por el usuario
- `recent_analysis.html` — listado de análisis recientes realizados por el usuario

Los ficheros estáticos sufrieron una reorganización equivalente. El CSS pasó a estar organizado por página y por sección dentro de `static/css/`, como se muestra en la Tabla 4.6:

Fichero	Responsabilidad
<code>home/hero.css</code>	Estilos de la sección principal de bienvenida
<code>home/examples.css</code>	Estilos de la sección de ejemplos
<code>home/how.css</code>	Estilos de la sección de funcionamiento
<code>home/lms.css</code>	Estilos de la sección de modelos compatibles
<code>home/metrics.css</code>	Estilos de la sección de métricas públicas
<code>home/cta.css</code>	Estilos de la sección de acceso
<code>assistant.css</code>	Estilos del asistente de generación
<code>history.css</code>	Estilos del historial del usuario
<code>edit.css</code>	Estilos de la página de edición del schema
<code>login.css</code>	Estilos de las páginas de login y registro
<code>navbar.css</code>	Estilos de la barra de navegación
<code>backgrounds.css</code>	Fondos y efectos visuales compartidos

Cuadro 4.6: Ficheros CSS tras la reorganización de estilos

El JavaScript siguió el mismo criterio con ficheros independientes por página en `static/js/`, como se muestra en la Tabla 4.7:

Fichero	Responsabilidad
<code>home.js</code>	Lógica interactiva de la home
<code>assistant.js</code>	Lógica del flujo del asistente
<code>edit.js</code>	Lógica de la edición del schema
<code>site_render.js</code>	Lógica de la vista previa del sitio generado

Cuadro 4.7: Ficheros JavaScript tras la reorganización

El `base.html` se actualizó para cargar las fuentes de Google Fonts necesarias para el nuevo diseño: Inter para el texto general, y Geist y Geist Mono para elementos de código y datos técnicos.

Esta reorganización, aunque invisible para el usuario final, facilitó enormemente el mantenimiento y la evolución de la interfaz en los sprints siguientes, ya que cualquier cambio en una sección concreta de una página solo requería tocar un fichero pequeño y bien delimitado.

4.6.3. Enriquecimiento de prompts

Tras unas cuantas pruebas, se llegó a la conclusión de un problema importante, podría ocurrir que un usuario no familiarizado con los prompts de IA no supiese cómo hacerlo adecuadamente, o que incluso lo dejase vacío. Para resolver este problema se creó el fichero `utils/llm/enrich_prompts.py`, encargado de analizar automáticamente los campos del dataset antes de construir el prompt y añadir contenido personalizado. Si el dataset tenía campos con imágenes se le indica al modelo que las usara de forma destacada. Si tenía precios, que los resalte visualmente. Si tenía fechas que las mostrara de forma clara. De esta forma se garantiza que el LLM siempre recibiera suficiente contexto para tomar decisiones razonadas. De todos modos si algo no estuviese en la mente del usuario, en el siguiente paso de editar el schema podría modificarlo añadiendo los campos necesarios o modificando los existentes.

4.6.4. Trazabilidad: GenerationLog

Anteriormente en las generaciones si algo fallaba no había forma de saber qué prompt se había enviado exactamente, qué había respondido el modelo ni si había habido reintentos. Para arreglar esto se creó el modelo **GenerationLog**, vinculado mediante una clave a su correspondiente **GeneratedSite**.

Cada llamada al LLM durante la generación de un proyecto producía un registro que almacenaba el paso concreto que se estaba generando, el modelo de LLM utilizado, el prompt de sistema y el prompt de usuario enviados, la respuesta en crudo devuelta por el modelo, los errores de consistencia detectados y si la llamada había necesitado un reintento. Esto permitía analizar a posteriori qué pasos fallaban con más frecuencia, qué modelos producían mejores resultados y qué tipos de datasets generaban más inconsistencias.

4.6.5. Verificación de consistencia

Otro problema frecuente era que el LLM no tenía suficiente contexto entre ficheros, principal causa de las alucinaciones del LLM. Ocurría lo siguiente: se definía un campo con un nombre en `models.py` y luego en los demás ficheros se referenciaba con ligeros cambios, provocando errores de ejecución. Para detectar estos casos antes de que el usuario desplegase el proyecto, se implementó el fichero `utils/llm/consistency_checker.py`.

Este fichero extraía mediante expresiones regulares los campos definidos en `models.py` y buscaba en los demás templates las referencias a esos campos. Cualquier referencia encontrada diferente que no correspondiera al campo real del modelo, se registraba como error de consistencia. Estos errores se guardaban en el campo `consistency_errors` del modelo `GenerationLog` correspondiente, permitiendo identificar qué pasos producían más inconsistencias y cómo se producían, para el consiguiente arreglo a futuro.

4.6.6. Panel de métricas

Para tener un uso más sencillo y accesible del contenido de `GenerationLog` se añadió una vista de métricas en `views/metrics.py`, accesible únicamente para usuarios con permisos de administrador. El objetivo de esta vista era tener visibilidad real sobre el comportamiento del generador, los fallos más recurrentes, qué modelos eran más óptimos para cada tipo de página, etc.

El panel mostraba las siguientes estadísticas agregadas:

- **Resumen general:** número total de registros en `GenerationLog`, sitios generados y análisis realizados hasta la fecha.
- **Desglose por paso:** para cada uno de los siete pasos del generador, se mostraba el número de llamadas realizadas, cuántas habían necesitado reintento y la tasa de reintento resultante. Esto permitía identificar qué pasos eran más problemáticos y priorizar las mejoras.
- **Distribución por modelo:** agrupación de las llamadas por modelo de LLM utilizado, lo que facilitaba comparar el rendimiento de distintos proveedores cuando se probaban configuraciones diferentes.

Esta información resultó útil para mejorar las siguientes fases donde cambios mínimos producirían grandes resultados.

4.7. Fase 6: Funcionalidades avanzadas

Una vez concluida una buena base del sistema de generación y despliegue, esta fase se dedicó a implementar funcionalidades que ampliaban las capacidades del sistema en cinco direcciones: versiones de proyectos generados, la gestión de usuarios de cada sitio, la ampliación de los formatos de entrada soportados, la mejora de la calidad visual de los templates mediante ejemplos few-shot, y la posibilidad de configurar un LLM personalizado por usuario.

4.7.1. Nuevos formatos de entrada: CSV y GeoJSON

Hasta este momento el módulo de ingestión solo soportaba JSON y XML. En esta fase se ampliaron los formatos añadiendo soporte para CSV y GeoJSON, que son dos de los formatos mas habituales en APIs publicas.

La detección del formato se añadió en el archivo existente **utils/ingest/parsers.py**.

El formato GeoJSON se detectaba antes que el formato JSON, ya que es un formato más restrictivo. Se realiza inspeccionando el campo `type` del objeto raíz: si era `FeatureCollection`, el fichero se procesaba como GeoJSON. Mientras que para el formato CSV se inspeccionaba la primera línea en busca de separadores de coma, o punto y coma, habituales en este tipo de ficheros.

El parseo de cada formato tenía su propia función:

- **CSV:** se utilizó la librería estándar `csv` de Python con detección automática del dialecto mediante `csv.Sniffer`. La lectura se limitaba a 2000 filas para evitar problemas de memoria con ficheros muy grandes.
- **GeoJSON:** cada `Feature` del `FeatureCollection` se aplanaba extrayendo sus propiedades como un diccionario plano, añadiendo además el tipo de geometría como campo `geometry_type`. De esta forma el resto del sistema podía tratar el resultado igual que cualquier otro JSON.

4.7.2. Ejemplos few-shot por tipo de sitio

Otro problema detectado en las generaciones era que los templates HTML producidos por el LLM tendían a ser genéricos y con poco estilo visual. Para mejorar la calidad del código generado se introdujo la carpeta `utils/llm/examples/`, con ejemplos de templates HTML reales por tipo de sitio: `blog.py`, `catalog.py`, `dashboard.py` y `portfolio.py`. Estos ejemplos se incluían en el prompt del paso 5 del generador como referencia de estilo, siguiendo la técnica de *few-shot prompting* [11], que consiste en proporcionar al modelo ejemplos concretos del resultado esperado para que los imite en lugar de inventar una solución desde cero. El resultado fue una mejora notable en la calidad visual y estructural de los templates generados.

4.7.3. Versionado de proyectos generados

Hasta este momento, generar o editar una API existente en el historial, causaba que los ficheros existentes se sobrescribieran, de esta forma era imposible tener un control de versiones o recurrir a una prueba anterior que fuese mejor. Para solucionar esto se creo el modelo **SiteVersion**, enlazado mediante una clave foránea a **GeneratedSite**, como se muestra en la Tabla 4.8. Cada version estaba nombrada con un numero incremental, y almacenaba una copia completa de los ficheros, una etiqueta descriptiva opcional y la fecha de creación del proyecto. Las versiones estaban ordenadas de forma más reciente a mas antigua, permitiendo asi al usuario consultar un historial funcional y la posibilidad de recuperar cualquier version anterior.

Campo	Tipo	Descripción
site	ForeignKey	Sitio generado al que pertenece la versión
version_number	IntegerField	Número incremental de la versión
project_files	JSONField	Copia completa de los ficheros del proyecto
label	CharField	Etiqueta descriptiva opcional
created_at	DateTimeField	Fecha de creación de la versión

Cuadro 4.8: Campos del modelo SiteVersion

4.7.4. Usuarios del proyecto

Cada sitio Django generado incluía su propio sistema de autenticación, con paginas de login y register. Para gestionar los usuarios de esos sitios desde la propia plataforma se creó el modelo **SiteUser**, vinculado a **GeneratedSite**, como se muestra en la Tabla 4.9. Cada usuario del sitio tenía su nombre, una contraseña y un rol. Este ultimo campo podria ser `super` para la creacion de un superusuario con acceso al panel de administracion Django del proyecto creado, o `normal` para un usuario con acceso solo al frontend. La combinacion de proyecto y nombre de usuario era única, evitando asi duplicados dentro del mismo proyecto.

Campo	Tipo	Descripción
site	ForeignKey	Sitio generado al que pertenece el usuario
username	CharField	Nombre del usuario del sitio generado
password	CharField	Contraseña del usuario
role	CharField	Rol del usuario: super o normal

Cuadro 4.9: Campos del modelo SiteUser

4.7.5. LLM local y catálogo de modelos disponibles

Hasta este momento el modelo de LLM utilizado para generar páginas era fijo, configurado en el fichero `.env` del servidor, sin que el usuario pudiera cambiarlo. En esta fase se implementaron dos cambios importantes respecto a esto.

En primer lugar, se creó el fichero `utils/llm/llm_catalog.py`, que definía un catálogo de modelos conocidos y recomendados. Cada entrada del catálogo incluía el identificador del modelo, su nombre legible, el proveedor, la URL base del API, una descripción y una lista de ventajas e inconvenientes. En este sprint el catálogo incluía tres modelos:

- **Llama 4 Scout** (Groq) — modelo rápido y eficiente, recomendado para generaciones ágiles con buena calidad de código
- **Llama 3.3 70B** (OpenRouter) — modelo más potente, recomendado para webs complejas que requieren mayor capacidad de razonamiento
- **Qwen3 Coder** (OpenRouter) — modelo especializado en código, especialmente bueno generando HTML y CSS limpio

En segundo lugar, se implementó un selector de modelo completo, accesible desde el inicio del proceso, justo antes de analizar la URL. El selector ofrecía tres modos de uso:

- **Predeterminado:** el sistema selecciona automáticamente el modelo más adecuado según el dataset analizado, buscando el equilibrio óptimo entre velocidad y calidad. Es la opción recomendada para la mayoría de casos.
- **Elegir modelo:** despliega el catálogo de modelos disponibles definido en `utils/llm/llm_catalog.py`, mostrando para cada modelo su nombre, proveedor, descripción y sus ventajas e inconvenientes.
- **Modelo personalizado:** utiliza el LLM configurado por el usuario en su perfil. Si el usuario no tiene ninguno configurado, esta opción aparece deshabilitada.

El catálogo cubría los casos más habituales, pero existía la necesidad de permitir que cualquier usuario pudiera conectar su propio proveedor de LLM, ya fuera uno privado, uno de pago con mejores capacidades o simplemente uno diferente a los disponibles en el catálogo. Para soportar esto se creó el modelo `UserProfile`, vinculado en relación `OneToOne` con el usuario de Django, como se muestra en la Tabla 4.10. El modelo almacenaba tres campos opcionales:

- **URL base del proveedor** (`custom_llm_base_url`): dirección del endpoint del proveedor OpenAI-compatible, por ejemplo `https://api.groq.com/openai/v1` para Groq o `https://openrouter.ai/api/v1` para OpenRouter.
- **Identificador del modelo** (`custom_llm_model`): nombre exacto del modelo tal y como lo espera el proveedor, por ejemplo `meta-llama/llama-3.3-70b-instruct:free`.
- **API key** (`custom_llm_api_key`): clave de acceso al proveedor, cifrada en base de datos mediante `EncryptedCharField` para que nunca se almacene en texto plano.

Campo	Tipo	Descripción
<code>user</code>	<code>OneToOneField</code>	Usuario de Django al que pertenece el perfil
<code>custom_llm_base_url</code>	<code>CharField</code>	URL base del proveedor OpenAI-compatible
<code>custom_llm_model</code>	<code>CharField</code>	Identificador del modelo del proveedor
<code>custom_llm_api_key</code>	<code>EncryptedCharField</code>	API key cifrada en base de datos

Cuadro 4.10: Campos del modelo `UserProfile`

4.8. Fase 7: Pulido y cierre

Esta última fase se dedicó a pulir el sistema en todos sus puntos débiles, que eran: mejorar la calidad visual de los sitios generados, consolidar la infraestructura de producción y completar los workflows de n8n que quedaban pendientes. Por infraestructura de producción se entienden los cambios que hacen que el proyecto sea útil fuera del entorno de desarrollo, como por ejemplo la migración a una base de datos más robusta como PostgreSQL para soportar accesos concurrentes de múltiples usuarios, la gestión automática de contenedores Docker activos o la monitorización diaria del sistema. Es decir, no se añadieron funcionalidades realmente nuevas, sino que se llevaron las existentes a un nivel de madurez adecuado para la entrega.

4.8.1. Sistema de presets visuales

Uno de los principales problema durante todo el desarrollo fue que los proyectos tendían a parecerse entre si visualmente, independiente del tipo de contenido de la API, del prompt del usuario y del sitio generado. Para resolverlo se creó el módulo **utils/llm/design**, organizado en tres ficheros con responsabilidades distintas. Estos ficheros eran `presets.py`, `snippets.py` y `theme_rules.py`.

El fichero `presets.py` define un catálogo de estilos visuales completos y muy distintos entre sí. Cada preset describe de forma exhaustiva la paleta de colores, el tipo de layout, la estética de los componentes y las palabras clave del prompt del usuario que deben activarlo. Los presets disponibles en esta fase se muestran en la Tabla 4.11:

Preset	Estética	Palabras clave activadoras
Neo Terminal	Cyberpunk, fondo negro, acento neón	terminal, dark, gaming, futurista
Bento Pop	Rejilla asimétrica, colores vivos	bento, mosaico, asimétrico
Quiet Editorial	Tipografía protagonista, estética revista	Por defecto para blogs
Magazine Split	Gran imagen dividida, tipografía impactante	Por defecto para catálogos
Glass Orbit	Glassmorphism, fondos oscuros, traslúcidos	Por defecto para portfolios

Cuadro 4.11: Presets visuales disponibles en el módulo de diseño

Cada tipo de sitio tiene asignado un preset por defecto que se aplica cuando el prompt del

usuario no contiene ningún estilo descrito, garantizando que el resultado siempre tenga una identidad visual coherente con el tipo de contenido.

El fichero `snippets.py` complementa los presets con fragmentos de HTML concretos y válidos para cada estilo: cards, heros y filas de listado con las clases Tailwind exactas que corresponden a cada preset. Estos snippets se incluyen en los prompts de generación de templates como ejemplos de referencia visual, asegurando que el LLM produzca código coherente con el estilo elegido.

El fichero `theme_rules.py` centraliza tokens visuales reutilizables como clases base para contenedores, cards, botones, badges y tipografía, junto con reglas críticas de uso de color para evitar que el LLM genere clases Tailwind inválidas, como `bg-#111111` en lugar de la sintaxis correcta `bg-[#111111]`.

4.8.2. Migración a PostgreSQL

Durante todo el desarrollo, WebBuilder había utilizado SQLite como base de datos, lo cual era suficiente para el entorno de desarrollo pero inadecuado para producción. En este sprint se realizó la migración a PostgreSQL, que ofrece mayor robustez, soporte para accesos concurrentes y mejor rendimiento con volúmenes de datos crecientes.

La configuración en `settings.py` se actualizó para leer los parámetros de conexión desde variables de entorno: nombre de la base de datos (`DB_NAME`), usuario (`DB_USER`), contraseña (`DB_PASSWORD`), host (`DB_HOST`) y puerto (`DB_PORT`, por defecto 5432). La configuración anterior de SQLite se migró a la nueva base de datos. Las migraciones existentes se aplicaron sobre la nueva base de datos, conservando los logs y métricas generados durante el desarrollo.

4.8.3. Workflow generation-done

Con la base de notificaciones ya preparada desde el fichero `utils/generator/notifications.py`, se implementó el workflow de n8n *generation-done*. Este workflow se activa mediante un webhook al que Django llama en cuanto termina la generación de un sitio, enviando un payload con los datos relevantes: email y nombre del usuario, título y tipo del sitio generado, URL de la API original, URL del preview si ya está desplegado, URL de descarga del ZIP, número de campos e items procesados, duración de la

generación en segundos y modelo de LLM utilizado. n8n recibe este payload y envía automáticamente un correo de notificación al usuario informándole de que su sitio está listo. La llamada al webhook se realizó de forma completamente transparente al flujo principal: si n8n no estaba disponible o la llamada fallaba, la excepción se capturaba silenciosamente mediante un bloque `try/except` y la generación continuaba con normalidad.

4.8.4. Workflow health-check

El workflow de *health-check* se configuró para ejecutarse automáticamente cada día mediante un nodo *Schedule Trigger* de n8n. Su función es verificar que el sistema sigue operativo realizando una petición HTTP a la URL principal de WebBuilder y comprobando que la respuesta es correcta. Si la verificación falla, n8n envía una alerta por correo al administrador del sistema con información sobre el error detectado. Este workflow fue especialmente útil durante las últimas fases de desarrollo para detectar caídas del servidor sin necesidad de monitorización manual.

4.8.5. Workflow auto-shutdown

El workflow de *auto-shutdown* resolvió un problema operativo importante: los contenedores Docker de los sitios generados permanecían activos indefinidamente aunque nadie los estuviera usando, consumiendo recursos del servidor. Este workflow se ejecuta de forma programada cada hora, consulta la lista de contenedores activos con el prefijo `wb_` y detiene aquellos que llevan inactivos más de un tiempo configurable. Esto permite mantener el servidor limpio y los recursos disponibles para nuevas generaciones sin intervención manual.

4.8.6. Mejora del workflow de despliegue

El workflow de despliegue que se había construido en el Sprint 4 carecía de un mecanismo centralizado de gestión de errores. Si cualquiera de los nodos fallaba, la petición de Django quedaba sin respuesta. En este sprint se añadió el nodo **Responder Error**, que actúa como rama de error compartida para todos los nodos del workflow. Cuando cualquier paso falla, este nodo captura el error y devuelve a Django una respuesta JSON estructurada con el código HTTP 500, un mensaje descriptivo del error y el paso en el que ocurrió. Esto permite que Django actualice

el estado del sitio a `error` y muestre un mensaje útil al usuario en lugar de dejar la operación colgada indefinidamente.

Capítulo 5

Experimentos y validación

En este capítulo se describe el proceso de validación de WebBuilder, comprobando que el sistema cumple con los objetivos planteados al inicio del proyecto. Se presenta la metodología seguida, los casos de uso reales generados con la plataforma, el análisis de los errores detectados durante el desarrollo y las métricas del sistema obtenidas tras varias semanas de uso.

5.0.1. Metodología de validación

La validación de WebBuilder se ha llevado a cabo mediante la generación de casos de uso reales, utilizando APIs públicas de distinta naturaleza y complejidad. El objetivo no ha sido medir la perfección de un resultado concreto, sino evaluar la capacidad del sistema para producir proyectos Django funcionales, desplegados y visualmente coherentes a partir de cualquier fuente de datos.

Para cada caso de uso se han evaluado los siguientes criterios:

- **Funcionalidad:** el proyecto generado arranca correctamente, carga los datos de la API y las páginas responden sin errores HTTP.
- **Coherencia del modelo de datos:** los campos detectados por el sistema corresponden a los datos reales de la API y el modelo Django generado los representa correctamente.
- **Calidad del código generado:** el código producido por el LLM es sintácticamente correcto, no contiene campos inexistentes y supera la verificación de consistencia del sistema.

- **Calidad visual:** el sitio generado tiene una apariencia coherente con el tipo de contenido, los presets visuales se aplican correctamente y la navegación entre páginas funciona.
- **Despliegue automático:** el workflow de n8n completa el proceso de build y arranque del contenedor Docker sin intervención manual, devolviendo una URL de previsualización accesible.

Además de los casos de uso, se ha realizado un análisis de los errores ocurridos en el proceso de desarrollo, categorizados por tipo y documentados con su causa, consecuencia y solución aplicada, que se recoge en la Sección ??.

5.0.2. Casos de uso

5.0.3. Análisis de errores

Durante el proceso de desarrollo y pruebas de WebBuilder se identificaron y documentaron de forma sistemática los principales errores que afectaban a la calidad de las generaciones. Estos errores se agrupan en dos categorías: limitaciones impuestas por los propios modelos de lenguaje y errores en el código generado por el LLM.

Errores de limitación del LLM

La primera categoría de errores está relacionada con los límites de uso de los proveedores de modelos, que condicionan directamente el comportamiento del sistema durante la generación.

Error HTTP 429 — Rate Limit Exceeded. El error más frecuente durante las pruebas fue la superación del límite de tokens por minuto del modelo. El prompt del paso de generación de templates es especialmente grande, ya que incluye el prompt enriquecido, los campos del modelo, los datos de ejemplo y un fragmento HTML de referencia. Cuando este prompt superaba los tokens disponibles en el minuto en curso, la llamada fallaba y el template afectado caía al fallback vacío, generando una página en blanco. La solución aplicada fue reducir el tamaño máximo del prompt enriquecido, eliminar los datos de ejemplo del paso de estructura de páginas y truncar el ejemplo HTML de referencia a 1500 caracteres.

Error HTTP 413 — Request Entity Too Large. Algunos modelos como `groq/compound` rechazan peticiones que superan un cierto tamaño en bytes antes de procesarlas, independiente-

mente del límite de TPM. Este error afectaba prácticamente a todos los pasos de la generación, dejando el proyecto resultante vacío o lleno de fallbacks. La solución fue descartar este modelo del catálogo recomendado y sustituirlo por `meta-llama/llama-4-scout-17b-16e-instruct`, que ofrece 30K TPM sin restricciones de tamaño de petición.

Timeout por `time.sleep` excesivo. Durante las pruebas se intentó evitar el error 429 aumentando el tiempo de espera entre llamadas al LLM hasta 60 segundos. El resultado fue que el servidor web cortaba la conexión por timeout antes de que terminara la generación. La conclusión fue que el sleep no debe superar los 20 segundos y que la solución correcta al problema de TPM es reducir el tamaño de los prompts, no aumentar la espera entre llamadas.

En la Tabla 5.1 se recoge la comparativa de modelos evaluados en Groq durante las pruebas, con sus límites y observaciones:

Modelo	TPM	RPD	Observaciones
groq/compound	70K	Sin límite	Límite de tamaño de request muy restrictivo. No recomendado.
openai/gpt-oss-120b	8K	200K	Buena calidad pero TPM muy bajo. Falla en prompts grandes.
llama-3.3-70b-versatile	12K	100K	Aceptable pero con menos margen.
llama-4-scout-17b-16e	30K	500K	Recomendado. Buen equilibrio calidad/límites.

Cuadro 5.1: Comparativa de modelos evaluados en Groq

Errores en el código generado

La segunda categoría agrupa los errores producidos por el propio LLM al generar el código del proyecto, independientemente de los límites del proveedor.

Acceso a subcampos de objetos anidados. Cuando el dataset contiene campos que son objetos anidados, como `origin: name: Earth''`, `url: ...''`, el LLM tiende a tra-

tarlos como objetos Django en el template usando la sintaxis `item.origin.name`. Sin embargo, el modelo Django almacena ese campo como un `CharField` simple con solo el valor del nombre. El resultado es un error en tiempo de ejecución al intentar acceder a `.name` sobre un string. Este error es especialmente difícil de detectar automáticamente porque el verificador de consistencia solo comprueba que el primer nivel de campo exista en el modelo.

URLs hardcodeadas incorrectas en templates. El LLM tiende a usar nombres de URL genéricos como `detail` o `catalog` en las etiquetas `url` aunque el prompt le indique los nombres exactos. Esto provoca errores `NoReverseMatch` en tiempo de ejecución. La solución aplicada fue añadir una validación en `check_django_syntax` que comprueba que todos los nombres de URL usados en los templates corresponden a URLs reales del proyecto.

`json.loads` sobre un dict ya parseado. Cuando el dataset tiene campos anidados, el LLM a veces genera en `load_data.py` llamadas a `json.loads()` sobre campos que ya han sido deserializados por `response.json()`. El resultado es un `json.JSONDecodeError` que hace que todos los items fallen silenciosamente y no se cargue ningún dato en la base de datos.

Variables de contexto inconsistentes entre view y template. En ocasiones el LLM usa nombres semánticamente descriptivos en la view, como `featured` o `recent`, en lugar del nombre estándar `items` que usan los templates. El resultado es que la página aparece vacía aunque haya datos en la base de datos. La solución fue reforzar en los prompts de generación que la variable del contexto debe llamarse siempre `items`.

Paginación no implementada en APIs paginadas. Cuando la API devuelve los datos paginados, el `load_data.py` generado por defecto solo hace una petición a la URL base, cargando únicamente la primera página de resultados. Esto puede suponer cargar 20 items de una API que tiene 800, sin que el sistema lo detecte como un error.

5.1. Métricas del sistema

WebBuilder incorpora un panel de métricas accesible únicamente para administradores que recopila datos en tiempo real sobre el funcionamiento del sistema. Esta sección presenta los resultados extraídos del panel tras el uso de la pagina, que permiten evaluar de forma objetiva tanto el rendimiento como la fiabilidad de la plataforma.

El panel se estructura en seis vistas independientes (Resumen, Actividad, LLM, Alertas, Usuarios y Generaciones), cada una enfocada en un aspecto distinto del sistema. Tanto los datos numéricos como las capturas que se muestran a continuación corresponden a una instantánea del sistema en el momento de redactar esta memoria, por lo que pueden no coincidir exactamente con los valores actuales en el momento de la lectura.

5.1.1. Resumen general

El sistema acumula 67 análisis de APIs realizados, de los cuales 61 han sido generados con éxito. Esto supone una tasa de éxito del análisis del 95.5 % y una conversión análisis-a-sitio del 91 %, lo que indica que el flujo completo desde la ingesta de datos hasta la generación final es estable y fiable. La tasa de reintentos del LLM se mantiene en el 0 %, dentro del rango esperado para los modelos seleccionados como recomendados tras los ajustes descritos en la Sección 5.1.3.

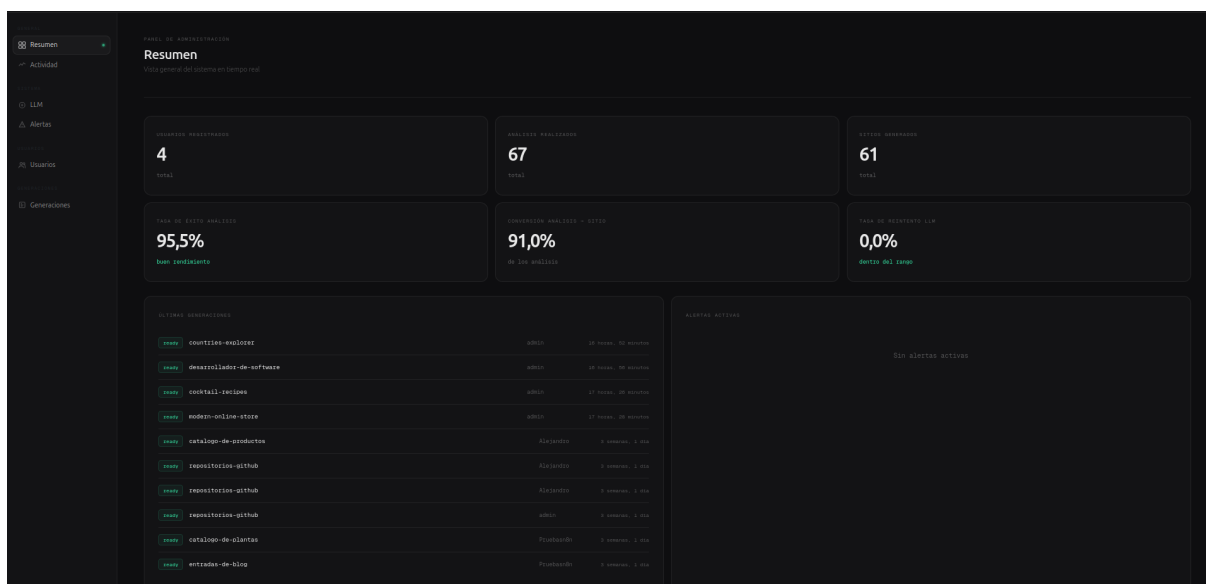


Figura 5.1: Panel de Resumen general del sistema

5.1.2. Actividad y patrones de uso

El panel de actividad muestra la evolución de los análisis y las generaciones en los últimos 30 días. Se observan picos de uso en los periodos de desarrollo de mejora de visualizacion, alcanzando hasta 15 análisis en un mismo.

El modelo llama-4-scout acumula la inmensa mayoría de las llamadas tras ser el modelo mas adecuado por su equilibrio entre calidad y límites de TPM, mientras que groq/compound fue descartado tras detectar el problema de tamaño máximo de petición.

La tasa de errores de consistencia es del 0 % en todos los modelos, lo que valida la eficacia del sistema de verificación de código generado descrito en la Sección ??.

El número medio de llamadas por sitio generado es de 7.5, coherente con los 7 pasos del generador más alguna llamada adicional de validación.

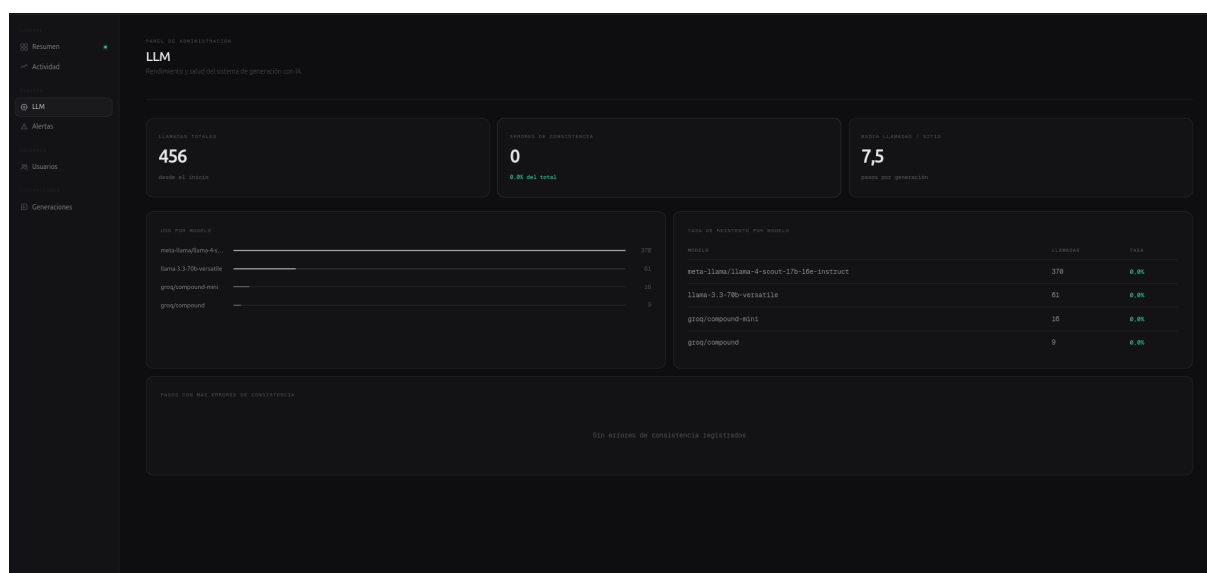


Figura 5.3: Panel de rendimiento de los modelos LLM

5.1.4. Alertas y estabilidad

El panel de alertas detecta incidencias del sistema como generaciones bloqueadas, análisis pendientes o errores recientes. En el momento de la captura no había alertas activas ni errores registrados en las últimas 24 horas.

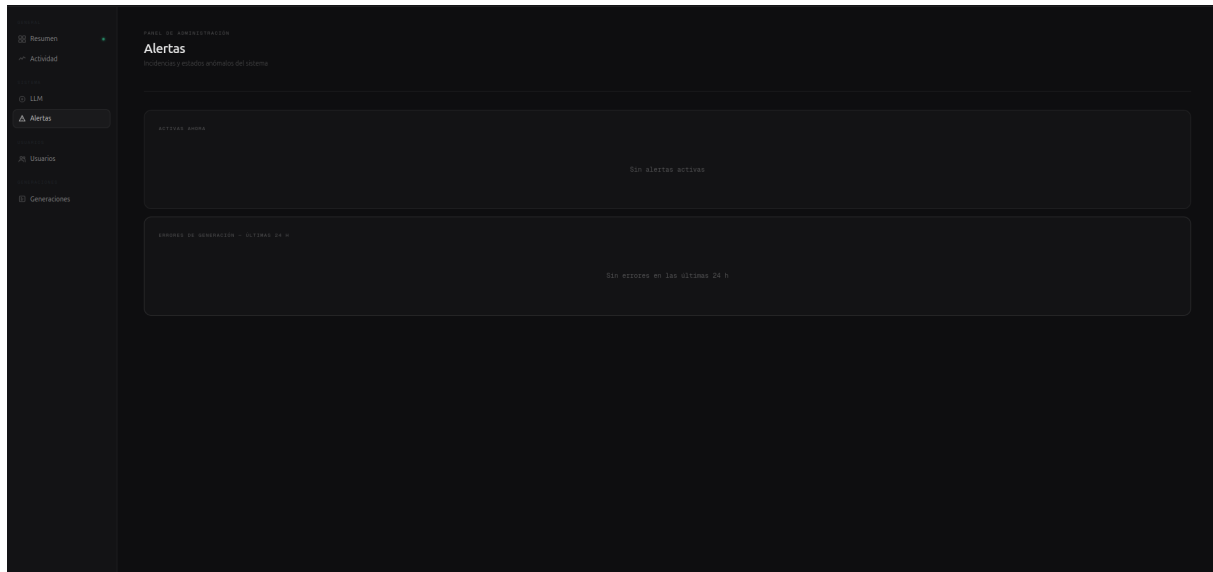


Figura 5.4: Panel de alertas del sistema

5.1.5. Usuarios y proyectos generados

La plataforma cuenta con 4 usuarios registrados, todos ellos con al menos un sitio generado. La distribución de actividad muestra que el usuario `admin` concentra 52 análisis y 47 sitios generados (lo esperable dado que es la cuenta de pruebas principal), seguido por `Alejandro` con 9 análisis y 9 sitios, y los usuarios `Pruebasn8n` y `pruebas2` con volúmenes menores asociados a pruebas específicas.

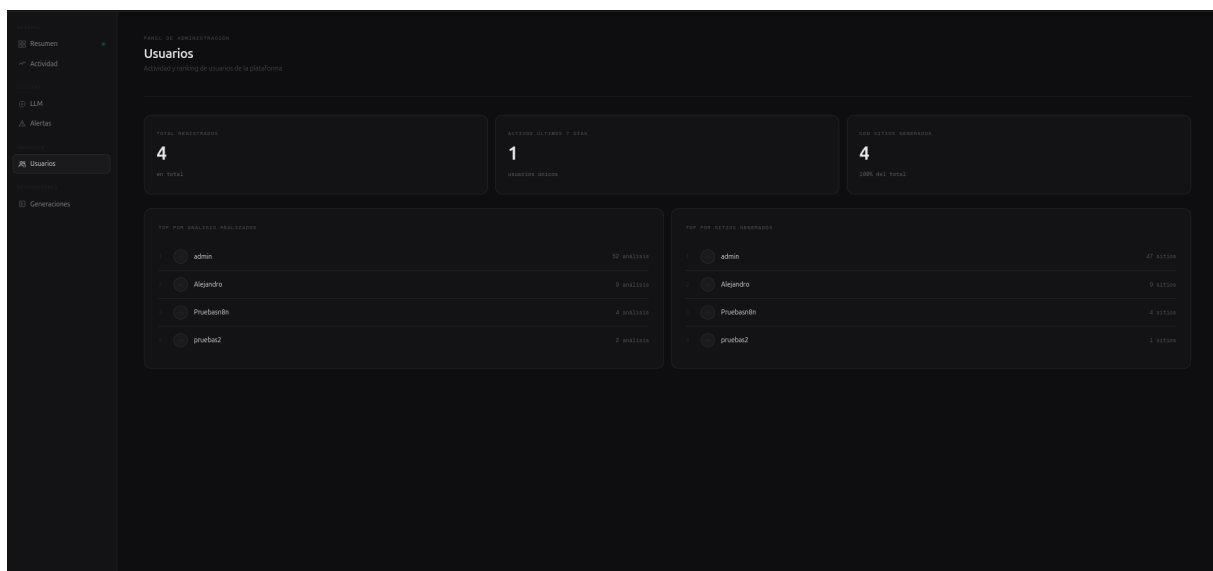
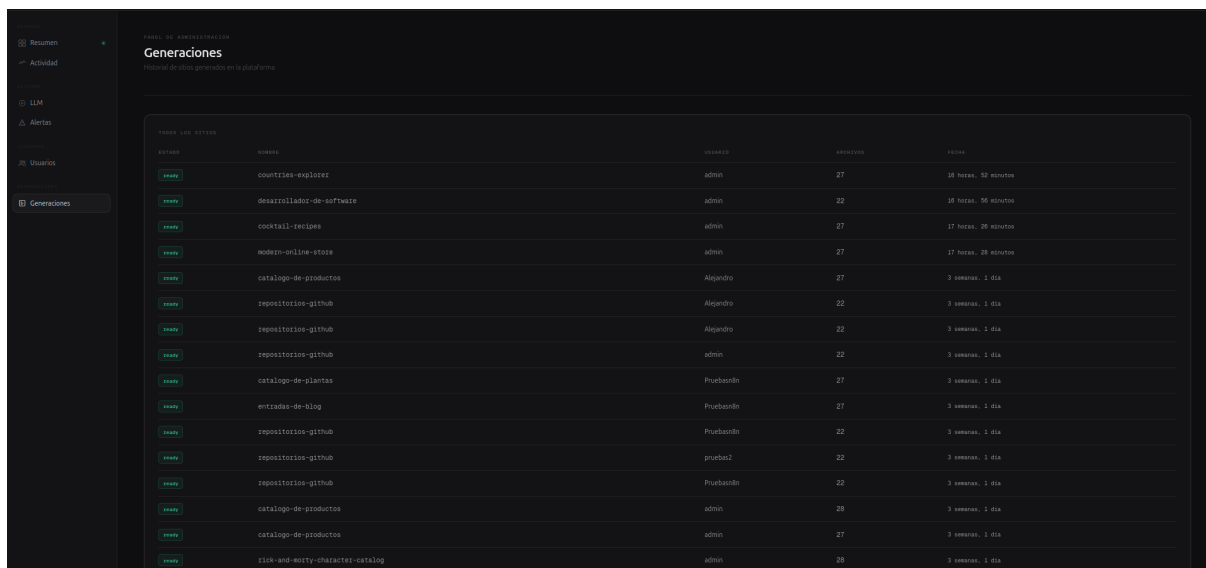


Figura 5.5: Panel de metricas sobre usuarios

El panel de generaciones muestra el listado completo de proyectos creados con su estado, nombre, usuario propietario, número de ficheros y antigüedad. Todos los proyectos listados se encuentran en estado *ready*, lo que indica que su contenedor Docker está disponible y accesible mediante su URL de previsualización.



ESTADO	NOMBRE	USUARIO	FICHEROS	EDAD
ready	countries-explorer	admin	27	16 horas, 52 minutos
ready	desarrollador-de-software	admin	22	16 horas, 56 minutos
ready	cocktail-recipes	admin	27	17 horas, 26 minutos
ready	modern-online-store	admin	27	17 horas, 28 minutos
ready	catalogo-de-productos	Alejandro	27	3 semanas, 1 día
ready	repositorios-github	Alejandro	22	3 semanas, 1 día
ready	repositorios-github	Alejandro	22	3 semanas, 1 día
ready	repositorios-github	admin	22	3 semanas, 1 día
ready	catalogo-de-plantas	Pruflasoft	27	3 semanas, 1 día
ready	entidad-de-ling	Pruflasoft	27	3 semanas, 1 día
ready	repositorios-github	Pruflasoft	22	3 semanas, 1 día
ready	repositorios-github	pruflasoft	22	3 semanas, 1 día
ready	repositorios-github	Pruflasoft	22	3 semanas, 1 día
ready	catalogo-de-productos	admin	28	3 semanas, 1 día
ready	catalogo-de-productos	admin	27	3 semanas, 1 día
ready	zick-and-mitty-character-catalog	admin	28	3 semanas, 1 día

Figura 5.6: Panel de historial de generaciones de la plataforma

Capítulo 6

Resultados

En este capítulo se incluyen los resultados de tu trabajo fin de grado.

Si es una herramienta de análisis lo que has realizado, aquí puedes poner ejemplos de haberla utilizado para que se vea su utilidad.

Capítulo 7

Conclusiones

En este capítulo se recogen las conclusiones obtenidas tras el desarrollo de WebBuilder. En primer lugar se evalúan los objetivos planteados al inicio del proyecto, analizando qué se ha logrado y qué limitaciones se han encontrado. A continuación se reflexiona sobre la aplicación de los conocimientos adquiridos durante el grado y las lecciones aprendidas a lo largo del desarrollo. Por último se proponen posibles líneas de trabajo futuro que podrían ampliar o mejorar el sistema en próximas iteraciones.

7.1. Consecución de objetivos

Cabe destacar que una de las principales limitaciones encontradas durante el desarrollo ha sido la calidad visual de los sitios generados. La calidad que producen modelos de pago de alto rendimiento como GPT-4 o Claude no es alcanzable con los modelos gratuitos disponibles en los proveedores utilizados, y por más refinamiento aplicado en los prompts y en el sistema de presets visuales, llegar a ese nivel de calidad resulta considerablemente complejo sin acceso a modelos más potentes. Esta limitación es inherente al estado actual de los modelos gratuitos y no a la arquitectura del sistema en sí.

- **Desarrollar un módulo de ingestión de datos.** Este objetivo se ha cumplido completamente. El módulo implementado en `utils/ingest/` es capaz de validar la URL introducida por el usuario, descargar el contenido de la API de forma segura y detectar automáticamente el formato de los datos. Los cuatro formatos contemplados en el objetivo, JSON, XML, CSV y GeoJSON, están soportados y producen una estructura Python

homogénea que el resto del sistema consume de forma transparente, soportados también a través de ficheros.

- **Integrar modelos de lenguaje para el análisis y la generación de código.** Este objetivo se ha cumplido completamente. El módulo `utils/llm/` implementa un cliente compatible con el estándar OpenAI que permite conectar con cualquier proveedor del mercado. Los modelos de lenguaje intervienen en seis de los siete pasos del generador, produciendo desde el modelo de datos Django hasta los templates HTML y el comando de carga de datos. Se han diseñado prompts específicos para cada paso tras muchas pruebas y refinamientos.
- **Implementar un generador de proyectos Django en siete pasos.** Este objetivo se ha cumplido completamente. El generador, implementado en `utils/generator/project_generator.py`, produce un proyecto Django completo y desplegable a partir del esquema validado por el usuario. Cada paso tiene su propio prompt, su propio fallback de emergencia y su registro en el modelo `GenerationLog`, garantizando que el resultado siempre sea desplegable incluso cuando el LLM produce una respuesta de baja calidad.
- **Automatizar el despliegue mediante Docker y n8n.** Este objetivo se ha cumplido completamente. El workflow `webbuilder-deploy` de n8n recibe la señal de Django mediante un webhook, construye la imagen Docker del proyecto generado, selecciona un puerto libre, arranca el contenedor y devuelve la URL de previsualización. Todo el proceso es transparente para el usuario, que recibe además una notificación por correo cuando su sitio está listo.
- **Diseñar un sistema de trazabilidad y métricas.** Este objetivo se ha cumplido completamente. El modelo `GenerationLog` registra cada llamada al LLM durante la generación, incluyendo el prompt enviado, la respuesta recibida, los errores detectados y si fue necesario un reintento. El verificador de consistencia `consistency_checker.py` detecta referencias incorrectas entre ficheros antes del despliegue. El panel de métricas, accesible para administradores, toma esta información y muestra los datos a modo de dashboard.
- **Ofrecer soporte multi-LLM y configuración personalizada.** Este objetivo se ha cum-

plido completamente. El catálogo de modelos definido en `utils/llm/llm_catalog.py` ofrece al usuario tres modelos preconfigurados con sus ventajas e inconvenientes. Además, el modelo `UserProfile` permite a cualquier usuario conectar su propio proveedor OpenAI-compatible introduciendo su URL base, el identificador del modelo y su clave de API, que se almacena cifrada en base de datos mediante `EncryptedCharField`.

7.2. Aplicación de lo aprendido

Durante el desarrollo de WebBuilder se han puesto en práctica numerosos conocimientos adquiridos a lo largo del Grado en Ingeniería Telemática. A continuación se detallan las asignaturas más directamente relacionadas con el proyecto:

1. **Programación de Sistemas de Telecomunicación:** Esta asignatura proporcionó la base de programación en Python que ha sido el lenguaje principal del proyecto. Los conocimientos adquiridos sobre estructuras de datos, funciones, módulos y manejo de ficheros han sido esenciales para desarrollar toda la lógica del backend, desde el módulo de ingestión hasta el generador de proyectos.
2. **Laboratorio de Bases de Datos:** Los conceptos de modelado relacional, consultas y gestión de datos aprendidos en esta asignatura han sido fundamentales para diseñar los modelos de Django, gestionar las migraciones y buscar información concreta en PostgreSQL.
3. **Aplicaciones Telemáticas:** Esta asignatura introdujo el desarrollo web con HTML, CSS y JavaScript, tecnologías que conforman toda la capa de presentación de WebBuilder. Además, los conceptos de contenedores y entornos de ejecución aislados trabajados en esta asignatura sentaron las bases para comprender y aplicar Docker en el despliegue de los proyectos generados.
4. **Servicios Telemáticos:** El framework Django, utilizado como núcleo del sistema, fue introducido en esta asignatura. El manejo de peticiones, respuestas, rutas y vistas que se aplica en WebBuilder bebe directamente de los conocimientos adquiridos aquí, así como la comprensión del modelo cliente-servidor sobre el que se construye toda la plataforma.

5. **Seguridad en Redes de Ordenadores:** Los conocimientos sobre seguridad y cifrado adquiridos en esta asignatura han sido aplicados directamente en la protección de las claves de API de los proveedores LLM, almacenadas cifradas en base de datos mediante `EncryptedCharField`, evitando que se guarden en texto plano.
6. **Sistemas Operativos:** Aunque esta asignatura se centró en programación en C y conceptos de bajo nivel como procesos, kernel y gestión de recursos, esos fundamentos han facilitado la comprensión del funcionamiento interno de Docker: la gestión de contenedores, la asignación de puertos y el aislamiento de procesos en el sistema anfitrión.

Este proyecto ha ayudado a integrar y aplicar de forma práctica los conocimientos adquiridos a lo largo del grado, demostrando que las competencias desarrolladas en cada asignatura no son independientes sino que se complementan y convergen en un proyecto real de ingeniería.

7.3. Lecciones aprendidas

El desarrollo de WebBuilder ha supuesto el aprendizaje de varias tecnologías que no formaban parte de la carrera. Entre ellas destacan:

1. **Automatización de flujos con n8n:** El uso de n8n para orquestar flujos de trabajo asíncronos ha sido un aprendizaje completamente nuevo. Aprender a diseñar workflows con nodos, configurar webhooks, gestionar errores en cada paso y conectar servicios externos.
2. **Integración de modelos de lenguaje mediante APIs:** Conectar el sistema a proveedores de LLM como Groq, OpenRouter o NVIDIA NIM, gestionar sus límites de uso, interpretar sus respuestas y manejar sus fallos, así como aprender a manejar el estándar OpenAI.
3. **Ingeniería de prompts:** El diseño, prueba y refinamiento de prompts efectivos necesarios durante el proyecto. Aprender a formular instrucciones precisas, cómo estructurar el contexto que se le pasa al modelo y cómo proteger el sistema frente a respuestas inesperadas ha requerido un proceso largo de experimentación.
4. **Tailwind CSS:** El uso de Tailwind como sistema de estilos para los proyectos generados, ha sido aprendido íntegramente durante el proyecto. Su compatibilidad natural con la generación por LLM lo han convertido en una herramienta indispensable de la generación.

5. **Gestión autónoma de un proyecto de software:** Llevar un proyecto de esta envergadura de forma completamente autónoma, realizando investigación, tomando decisiones de arquitectura, investigando y añadiendo funcionalidades, organizando y realizando refactors cuando era necesario.
6. **Redacción técnica con L^AT_EX:** La elaboración de esta memoria ha supuesto aprender a trabajar con L^AT_EX desde cero, incluyendo la gestión de referencias, figuras, tablas y la estructura general de un documento técnico académico.

7.4. Trabajos futuros

Aunque WebBuilder cumple con los objetivos planteados, existen alguna líneas de mejora y ampliación. A continuación se proponen las mas relevantes, tanto mejoras técnicas del sistema como posibles funcionalidades.

1. **Mejora de la calidad visual de los sitios generados:** La principal línea de trabajo futuro es mejorar la calidad visual de los proyectos generados. Esto pasa por incorporar modelos de mayor capacidad y ampliar la librería de ejemplos few-shot con más variantes de estilo.
2. **Validación automática del código generado:** Actualmente el sistema detecta inconsistencias entre ficheros pero no ejecuta el proyecto antes de desplegarlo. Una mejora sería lanzar el proyecto en un contenedor efímero tras la generación, verificar que responde HTTP 200 y solo entonces marcarlo como listo, cerrando el ciclo de calidad de forma completamente automática.
3. **Soporte para más formatos de entrada:** Además de URLs de APIs públicas, sería interesante permitir al usuario usar APIs o ficheros en cualquier formato, ampliando así las fuentes de datos que el sistema puede procesar.
4. **Comparador de modelos LLM:** Permitir lanzar la misma generación con dos modelos distintos en paralelo y comparar los resultados sería una funcionalidad muy potente, tanto para el usuario final como para el análisis interno del sistema.
5. **Galería pública de proyectos generados:** Implementar una sección donde los usuarios puedan marcar sus proyectos como públicos y compartirlos en una galería con el tipo de

sitio, la API de origen y una captura visual. Convirtiendo así WebBuilder en una herramienta social añadiendo interactividad y aumentando su potencial.

6. **Sistema de autenticación con proveedores externos:** Incorporar la posibilidad de iniciar sesión con Google u otros proveedores mediante OAuth, simplificando el acceso a la plataforma y mejorando la experiencia de usuario.

Apéndice A

Manual de usuario

Esto es un apéndice. Si has creado una aplicación, siempre viene bien tener un manual de usuario. Pues ponlo aquí.

Bibliografía

- [1] Django Software Foundation. Django documentation — django overview, 2024. Consultado en mayo de 2026.
- [2] Docker Inc. Docker documentation, 2024. Consultado en mayo de 2026.
- [3] Groq. Groq documentation, 2024. Consultado en mayo de 2026.
- [4] Mozilla Developer Network. Css: Cascading style sheets, 2024. Consultado en mayo de 2026.
- [5] Mozilla Developer Network. Html: Hypertext markup language, 2024. Consultado en mayo de 2026.
- [6] Mozilla Developer Network. Javascript, 2024. Consultado en mayo de 2026.
- [7] n8n GmbH. n8n documentation, 2024. Consultado en mayo de 2026.
- [8] NVIDIA. Nvidia nim documentation, 2024. Consultado en mayo de 2026.
- [9] OpenAI. Introducing ChatGPT, 2022. Consultado en mayo de 2026.
- [10] OpenAI. Openai api reference, 2024. Consultado en mayo de 2026.
- [11] OpenAI. Prompt engineering guide, 2024. Accedido: abril 2026.
- [12] OpenRouter. Openrouter documentation, 2024. Consultado en mayo de 2026.
- [13] Python Software Foundation. History and license, 2024. Consultado en mayo de 2026.
- [14] K. Sookocheff. How do webhooks work?, 2019. Consultado en mayo de 2026.
- [15] SQLite Consortium. Sqlite documentation, 2024. Consultado en mayo de 2026.

- [16] Stack Overflow. Stack overflow developer survey 2025 — technology, 2025. Consultado en mayo de 2026.
- [17] Tailwind Labs. Tailwind css documentation, 2024. Consultado en mayo de 2026.
- [18] The PostgreSQL Global Development Group. Postgresql documentation, 2024. Consultado en mayo de 2026.
- [19] A. Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [20] World Wide Web Consortium. World wide web consortium (W3C), 2024. Consultado en mayo de 2026.
- [21] P. Zhao et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.